

Dynamic Monitoring of Radiological Pollution through Deep Learning

Anomaly classification in noisy gamma-dose time series

Pedro

CentraleSupélec — Pôle Projet

Supervised by ELISABETH LAHALLE (L2S, CentraleSupélec)

May 2026

Abstract

Continuous monitoring of radiological pollution relies on environmental surveillance beacons that record the rate of gamma photons reaching a scintillation detector. The recorded signal is heavily contaminated by sensor noise and slow baseline drift, and the events of interest — spikes, plateaus, ramp-ups, ramp-downs — are sparse and exhibit a wide range of shapes, amplitudes and durations. The goal of this *Pôle Projet* at CentraleSupélec was to design, train, and evaluate deep neural networks that can classify these events on streaming gamma-dose recordings.

The project ran in two iterations. The *first* (v1) calibrated a white-Gaussian-plus-Ornstein–Uhlenbeck background model from four months of real 2015 recordings and injected six hand-designed anomaly templates densely into a 10^7 -sample synthetic signal; three PyTorch architectures (a 1D-CNN, a 1D residual network, and a hybrid CNN+Transformer) were trained on this dataset. On the held-out synthetic test segment, all three models cleared 90% event-level macro F1, with the residual network reaching a flawless 100%. On the *real* 2015 data, however, all three collapsed in different ways — continuous false-positive plateaus, oscillating predictions, single-class fixation. The synthetic–real distribution shift turned out to be much larger than the noise model had suggested.

The *second* iteration (v2) re-analysed the background process, discovered that the high-frequency residual is well described by an AR(1) Gaussian rather than IID white noise ($\varphi \approx 0.86$, $\sigma_\epsilon \approx 1.12$), hand-labelled the seventeen plausible events visible across the four months, and re-tuned the generator to (i) reproduce the AR(1) noise texture, (ii) match the empirical anomaly amplitude and duration ranges, (iii) lower the injection density from ~ 7 to ~ 1.5 events per 10k samples, and (iv) prune the template family to the three shapes (`bell`, `fast_ascend`, `fast_descend`) that actually occur in the labels. Retrained on this corpus, the same three architectures now *commit* to the normal class on quiet stretches of the real signal and fire only where a visible event is present.

This report documents both iterations: the v1 retrospective, the re-analysis that exposed its blind spots, the design of v2, and the v2 results on synthetic data, robustness sweeps, and real recordings.

Keywords: time series classification, 1D convolutional neural networks, residual networks, self-attention, transformer encoder, radiological monitoring, synthetic data generation, AR(1) noise, distribution shift.

Contents

1	Introduction	9
1.1	Context: continuous radiological surveillance	9
1.2	Problem statement	9
1.3	Why deep learning?	10
1.4	The two iterations	10
1.5	Contributions	10
1.6	Report organisation	11
2	Theoretical background	12
2.1	Framing: time-series classification by windowing	12
2.2	Convolutional layers and supporting normalisations	13
2.2.1	The 1D convolution operator	13
2.2.2	Pointwise nonlinearity: ReLU	14
2.2.3	Pooling	14
2.2.4	Batch normalisation	14
2.2.5	Group normalisation	14
2.3	Residual connections	15
2.4	Attention and the Transformer encoder	15
2.4.1	Self-attention	15
2.4.2	Multi-head attention	16
2.4.3	Positional encoding	16
2.4.4	The Transformer encoder block	16
2.5	Training fundamentals	17
2.5.1	Cross-entropy loss	17
2.5.2	Class weighting	17
2.5.3	Adam optimiser	17
2.5.4	Learning rate scheduling	18
2.5.5	Early stopping	18
2.5.6	Temporal train/validation/test split	18
2.5.7	Window normalisation	18

3	Real 2015 data and the v1 noise model	19
3.1	Real 2015 gamma dose data	19
3.1.1	Source	19
3.1.2	Statistical characterisation	20
3.2	v1 noise model: Gaussian fluctuations on an OU baseline	22
4	The v1 synthetic generator	23
4.1	Anomaly templates (v1)	23
4.1.1	Stochastic deformation of a template	24
4.2	The v1 generator algorithm	25
4.3	The v1 dataset used in this report	25
4.4	Two v1 design choices that will matter later	26
5	Architectures and training pipeline	27
5.1	Architecture A — baseline 1D-CNN	27
5.2	Architecture B — 1D Residual Network	28
5.3	Architecture C — CNN+Attention	29
5.4	Side-by-side comparison	29
5.5	Training pipeline	30
5.5.1	Data preparation	30
5.5.2	Optimisation	30
5.5.3	Class-weighting in v1 vs v2	30
5.5.4	Selection criterion	31
6	v1: synthetic results and real-data failure	32
6.1	Synthetic-test performance	32
6.1.1	Training dynamics	32
6.1.2	Event-level evaluation methodology	33
6.1.3	v1 event-level results	33
6.2	v1 robustness study	34
6.3	v1 on real data: three architecture-specific failures	34
6.4	Five causes of the v1 failure	36
6.5	What to do about it	37
7	Re-analysis: AR(1) noise and hand labels	38
7.1	Methodology	38
7.2	High-frequency noise is AR(1), not white	39
7.3	AR-order selection: is AR(1) enough?	40
7.3.1	Stationarity	41
7.4	Baseline drift: linear, not mean-reverting	42

7.5	Hand labels: seventeen plausible events	42
7.5.1	Empirical event distributions	44
7.5.2	Shape assignment	45
7.5.3	Polarity	45
7.6	Targets for the v2 generator	45
8	The v2 synthetic generator	47
8.1	What changed, item by item	47
8.2	The v2 generator algorithm	49
8.3	The v2 dataset used in this report	49
8.4	Acceptance test	50
8.5	What v2 still does not fix	51
9	v2: synthetic results	52
9.1	Training dynamics	52
9.2	Window-level test performance	53
9.3	Event-level results	53
9.4	Cost–performance comparison	54
10	v2 robustness study	56
10.1	Protocol	56
10.2	Results	57
10.3	v1 vs v2 robustness in one figure	57
10.4	Discussion	58
11	v2 inference on real 2015 data	60
11.1	v2 inference on the first month	60
11.2	Side-by-side comparison on ResNet	62
11.3	Caveats	62
12	Conclusion and future work	64
12.1	Summary of contributions	64
12.2	Architecture recommendation	64
12.3	Limitations	65
12.4	Future work	65
12.4.1	Expert labelling of real events	65
12.4.2	Self-supervised pre-training on the real signal	65
12.4.3	Longer windows and stronger attention	66
12.4.4	An “unknown but not Normal” class	66
12.4.5	Explainability with Grad-CAM and attention visualisation	66

12.4.6	Multi-beacon and multi-modal extensions	66
12.5	Final words	66
A	Appendices	68
A.1	Anomaly template CSV format	68
A.2	Hyperparameter reference	68
A.2.1	Data generator	68
A.2.2	Training	69
A.2.3	Inference and event extraction	69
A.3	Reproducing the figures in this report	70
A.4	Software environment	71

List of Figures

2.1	A pre-LN Transformer encoder block. Each sub-layer is wrapped in a residual connection; LayerNorm is applied before the sub-layer (pre-norm).	17
3.1	The four months of real gamma-dose recordings. Each panel is a continuous $\sim 40\text{k}$ -sample segment at one sample per minute. The signal is centred around ~ 99 counts with a high-frequency dispersion of a few counts and slow drifts of a few units; a small number of sharper excursions are visible.	20
3.2	A 10 000-sample snapshot of the first month decomposed into a slowly-varying baseline (top, with the MA-480 baseline in red) and zero-mean residuals (bottom).	21
3.3	Left: empirical distribution of the raw gamma dose values per month. The distributions are unimodal and only slightly skewed, supporting a Gaussian approximation of the high-frequency noise. Right: residual autocorrelation (after MA-480 baseline removal). The non-trivial decay over ~ 30 lags is the v1 noise model’s biggest oversight; we will come back to it in chapter 7.	22
4.1	The six v1 anomaly templates, drawn in normalised time and amplitude. Each template is encoded as a small list of keypoints (red dots) connected by linear, exponential or half-cosine (“bell”) segments.	24
4.2	Five stochastic realisations of each v1 template with the parameter ranges of eq. (4.1). The qualitative shape is preserved, but no two realisations are identical.	25
4.3	First 30 000 samples (of 10^7) of the v1 synthetic dataset. Anomaly spans are coloured by class. Note the dense injection: roughly four visible bumps in a 30k-sample slice.	26
5.1	Sliding-window construction on a fragment of the v2 synthetic signal. Each coloured bar represents one 512-sample training window; red bars indicate windows that capture $\geq 30\%$ of an underlying event’s samples and so inherit its class label (event-coverage rule, section 2.1).	30
5.2	Left: window-level class distribution at training, for v1 (seven classes, blue) and v2 (four classes, green), against the (estimated) real deployment prior in red ($\sim 99\%$ Normal). v1 has been trained with a prior that is roughly the <i>opposite</i> of the deployment prior. v2 closes the gap to within a factor of three. Right: cross-entropy class weights used in each iteration. v2 strongly down-weights Normal ($w_0 = 0.27$ vs ~ 9 on the anomaly classes) because the class imbalance in v2 is also much larger than in v1.	31
6.1	v1 training dynamics. Left: train (dashed) and validation (solid) loss for the three models. Right: validation macro F1 (window level). All three models converge within 35 epochs and benefit from one or two learning-rate reductions.	32

6.2	v1 event-level confusion matrices on the synthetic test segment. Cell colour is the row-normalised recall; the integer inside each cell is the raw event count. The residual network achieves a perfect classification; the attention model misses three events; the baseline 1D-CNN misses four events.	33
6.3	v1 robustness curves. Left: macro F1 vs. background-noise multiplier. Right: macro F1 vs. anomaly-shape deformation multiplier. The dotted line marks 50% F1. The 1D-CNN drops sharply to a $\sim 7\%$ plateau under noise — the hallmark of a classifier that has lost confidence in its features and collapses to a single class. The residual and attention models degrade much more gracefully.	34
6.4	v1 inference on the first 10 000 samples of February 2015. Blue: raw gamma dose (left axis). Coloured lines: per-class confidence smoothed with a 30-sample rolling mean (right axis, 0–100%). The grey curve is the Normal Background confidence; the other colours are the six anomaly classes.	35
7.1	Diagnostics on the masked residual of February 2015. Top-left : marginal histogram, near-perfectly fit by $\mathcal{N}(0, 2.21)$. Top-right : Q-Q plot vs. Normal, on the diagonal. Bottom-left : autocorrelation function on the clean segment — $\text{ACF}(1) = 0.864$, decaying smoothly to zero by lag ~ 30 . Bottom-right : rolling standard deviation, stationary around the global value.	39
7.2	AR(1) innovation diagnostics, February 2015. Top-left: histogram + Gaussian fit. Top-right: Q-Q vs. Normal. Bottom-left: ACF of innovations, with bars inside or barely outside the $\pm 2/\sqrt{N}$ band. Bottom-right: Ljung–Box summary.	40
7.3	Empirical residual periodogram (grey scatter + blue smoothed line) against the theoretical PSDs of AR(p) fits. AR(1), AR(2) and AR(5) produce essentially indistinguishable theoretical spectra, all of which match the empirical periodogram across five decades of frequency.	41
7.4	Per-quarter AR(1) parameter estimates. Left : φ , stable around ~ 0.87 except for an early-month excursion in October likely caused by a few unmasked outliers. Right : innovation std σ_ε , stable around ~ 1.12 . The dashed lines mark the values used in the v2 generator.	41
7.5	The four months of real 2015 data with hand-labelled event spans (orange). Across the four months there are $3 + 3 + 8 + 3 = 17$ labelled events.	43
7.6	Zoomed view around each of the eight hand-labelled events in June 2015 (± 600 samples of context). The orange band marks the labelled interval; coloured lines are MA-30 (red) and MA-120 (green) smoothings. The first four events are very short (30–55 samples) sharp spikes; the last four are longer (138–281 samples), with a mix of bell and asymmetric shapes.	44
7.7	Empirical event statistics from the seventeen hand labels. Left : durations span [32, 1014] samples, with most events below 400. Right : peak amplitudes (above the MA-4320 local baseline) span [11.2, 36.8] counts. The red dashed lines mark the v2 generator’s [30, 1000]-sample and [15, 40] count bounds.	45
8.1	Left : 600 samples of IID Gaussian noise ($\sigma = 2.30$, v1) and of AR(1) noise ($\varphi = 0.86$, $\sigma_\varepsilon = 1.12$, v2) sharing the same marginal standard deviation. The AR(1) trace is visibly smoother on the few-sample scale. Right : the corresponding empirical ACFs, alongside the real-data residual ACF (black). The v2 noise matches the real ACF; the v1 noise does not.	47

8.2	The three v2 templates — the only shapes that have an empirical counterpart in the hand-labelled events of section 7.5.	48
8.3	Five stochastic realisations of each v2 template, drawn with log-uniform amplitudes in $[15, 40]$ counts and log-uniform durations in $[30, 1000]$ samples.	49
8.4	First 100 000 samples (of 10^7) of the v2 synthetic dataset. Anomaly spans are coloured by class. Note the sparse injection compared to fig. 4.3: ~ 15 events in this view, against four in the v1 30k-sample preview.	50
8.5	v1 vs v2 synthetic data on the same window size. v1 has four visible bumps; v2 has zero. The marginal noise standard deviation is the same ($\sigma \approx 2.3$), but the local texture of v2 is smoother because of the AR(1) memory.	50
9.1	v2 training dynamics. Left: train (dashed) and validation (solid) loss. Right: validation macro F1 (window level).	52
9.2	v2 event-level confusion matrices on the synthetic test segment. The leftmost column of each panel is the “missed event” column — a true event that has no overlapping prediction. No event is missed by any of the three models, including the four shortest ones in the segment (26, 28, 30 and 45 samples, near the lower bound of the v2 duration range). The 1D-CNN and CNN+Attention classify all 15 events correctly; the ResNet has a single off-diagonal entry — one 61-sample <code>bell</code> predicted as <code>fast_ascend</code>	53
9.3	v2 cost–performance trade-off. Left: parameter count (22.6k, 74.5k and 109.5k for the 1D-CNN, ResNet and CNN+Attention respectively). Centre: total training time on a single RTX 5060 Laptop GPU (the 1D-CNN ran the full 35 epochs in ~ 3.7 min, the CNN+Attention stopped early at epoch 21 after ~ 4.8 min, the ResNet at epoch 26 after ~ 5.3 min). Right: event-level macro F1 on the synthetic test segment. The 1D-CNN and the CNN+Attention reach a perfect 1.00; the ResNet sits at 0.92 because of the single misclassification discussed above.	55
10.1	v2 robustness curves. Left: event-level macro F1 as a function of the background-noise multiplier. Right: macro F1 as a function of the anomaly-shape deformation multiplier. The dotted line marks the 50% threshold.	57
10.2	Robustness curves of v1 (top row) and v2 (bottom row) on the same axes. The vertical position of the curves is not directly comparable between the two iterations — v1 reports macro F1 over six classes, v2 over three — but the qualitative behaviour is. The collapse of the 1D-CNN at high noise is much more brutal in v1 (drop to $\sim 7\%$) than in v2 (gentler drop to $\sim 4\%$ by the end of the sweep), and the residual / attention models are qualitatively similar in both iterations.	58
11.1	v2 inference on the first 10 000 samples of February 2015. Blue: raw gamma dose (left axis). Grey: smoothed Normal Background confidence. Coloured lines: the three anomaly classes (right axis, 0–100%). Compared to fig. 6.4, the grey background-confidence curve is near 100% throughout the quiet regions and only drops where the raw signal contains a visible bump.	61
11.2	ResNet 1D on the first 10 000 samples of February 2015. Top: v1, six classes, training on white-noise dataset — sustained false-positive plateaus across the entire window. Bottom: v2, four classes, training on AR(1) dataset calibrated to the hand labels — the model sits on Normal on the background and fires only at the labelled event around sample 3 500.	62

Chapter 1

Introduction

1.1 Context: continuous radiological surveillance

Environmental monitoring beacons measure the local radiation level by counting the gamma photons reaching a scintillation detector in a fixed sampling interval — typically one minute. The resulting time series, called the *gamma dose*, reflects the natural radioactive background, modulated by atmospheric conditions (temperature, humidity, pressure) and, hopefully rarely, by accidental or deliberate radiological events: dispersal of a radioactive plume, a leak from a nearby facility, the passage of a contaminated source.

The operational goal of a network of such beacons is to detect and classify those rare events *as early as possible* so that mitigation decisions can be taken with confidence. Three properties make this task fundamentally harder than industrial anomaly detection in mechanical or financial time series:

1. **High noise-to-signal ratio.** The sensor statistics produce a high-frequency dispersion that is comparable in magnitude to many anomalies of interest.
2. **Slow baseline drift.** The mean radiological background is not stationary — it wanders slowly with weather and seasonal factors, so an anomaly cannot be defined by a fixed threshold on the raw signal.
3. **Class diversity.** Different physical phenomena leave different temporal signatures: a sudden spike (a passing source), a slow ramp-up (an approaching plume), a sustained plateau (a deposited contamination), and so on. A meaningful response requires not just detection but classification.

1.2 Problem statement

The project, supervised by Élisabeth Lahalle at the *Laboratoire des Signaux et Systèmes* (L2S, CentraleSupélec), formulates the problem as a **streaming multi-class classification task** on a univariate gamma-dose time series. Given a sliding window of recent samples, the system must output one class label per window. The label set evolved between the two iterations of the project:

v1 (seven classes)	Normal		bell		bell_sq		fast_ascend		fast_descend		M		square
v2 (four classes)	Normal		bell		fast_ascend		fast_descend						

The reasons for shrinking the class set in v2 will become clear in chapters 7 and 8: only the three retained shapes actually appear in the seventeen plausible events that we hand-labelled on the real recordings.

1.3 Why deep learning?

A classical signal-processing pipeline — baseline removal followed by a hand-tuned shape-matching template bank — would in principle suffice on clean shapes, but it scales poorly to:

- *Shape variability*: real events do not match a template perfectly; their amplitude, duration and time-warping vary.
- *Noise*: at the noise levels observed in the real data, cross-correlation with a template becomes unreliable.
- *Class proliferation*: adding a new event class requires redesigning the bank.

Deep neural networks learn their own features from data, and the convolutional and attention building blocks of modern architectures are very effective at recognising local and global patterns in 1D signals. The reference project brief¹ explicitly suggests evaluating convolutional and attention layers, which we did.

1.4 The two iterations

This report covers two iterations of the same end-to-end pipeline.

Iteration 1 (v1). A noise model fitted on the four months of 2015 data — IID Gaussian high-frequency component on an Ornstein–Uhlenbeck baseline — and six hand-designed anomaly templates, injected at a density of ~ 7 events per 10k samples into a 10^7 -sample synthetic signal. Three architectures of increasing complexity (1D-CNN, ResNet 1D, CNN+Attention) were trained on this corpus. On the synthetic test set the system worked very well: all three architectures cleared a 90% event-level macro F1, the residual network reached 100%. But on the real recordings the system failed dramatically, in three architecture-specific ways.

Iteration 2 (v2). A re-analysis of the real background revealed that the high-frequency residual is autocorrelated rather than white, well described by an AR(1) process with $\varphi \approx 0.86$. A round of hand-labelling identified 17 plausible events across the four months, from which we measured empirical event amplitudes (log-uniform [15, 40] counts), durations (log-uniform [30, 1000] samples), and a template breakdown (only `bell`, `fast_ascend`, `fast_descend` appear). The generator was re-tuned accordingly — AR(1) noise, sparser injection (~ 1.5 events per 10k samples), three templates, absolute amplitude/duration scales — and the same three architectures were re-trained on the new corpus. On the real recordings, the v2 models sit on the `Normal Background` class for the quiet stretches and fire only where a visible event is present.

1.5 Contributions

Concretely, this project produced:

¹See `biblio/projet.pdf`.

1. A **statistical characterisation** of four months of real 2015 gamma-dose recordings, in two passes of increasing depth: a v1 fit of an IID-Gaussian-plus-OU model, and a v2 reanalysis that establishes the AR(1) structure of the high-frequency residual (chapters 3 and 7).
2. A **set of seventeen hand labels** of plausible real events, with empirical amplitudes, durations and shape assignments (section 7.5).
3. Two generations of a **synthetic dataset generator**: v1 (white noise, six templates, dense injection) and v2 (AR(1) noise, three templates, sparse injection, calibrated against the labels). Both produce a 10^7 -sample labelled signal in constant memory (chapters 4 and 8).
4. Three **PyTorch architectures** of increasing complexity — a baseline 1D-CNN ($\sim 23k$ params), a residual network ($\sim 75k$), and a hybrid CNN+Transformer with multi-head self-attention ($\sim 110k$) — all sharing the same I/O contract (chapter 5).
5. A **training and event-level evaluation pipeline** bridging sample-wise classification accuracy and operational event-detection metrics (chapter 5).
6. Quantitative **synthetic-test results** for v1 and v2, and a **robustness study** that sweeps background noise and shape deformation independently for both (chapters 6, 9 and 10).
7. Application of both v1 and v2 models to the real 2015 data, with a careful **diagnostic analysis** of where v1 fails and how v2 closes the gap qualitatively (chapters 6 and 11).

1.6 Report organisation

Chapter 2 provides the theoretical background — convolutions, residual learning, attention mechanisms, and the relevant training machinery — with enough mathematical detail to make the architectures self-contained for a reader without a deep-learning background. Chapter 3 describes the real 2015 recordings and the v1 noise model that the project initially fitted to them. Chapter 4 presents the v1 synthetic generator; chapter 5 the three networks shared between v1 and v2; chapter 6 the v1 synthetic-test results and the real-data failure that motivated v2. Chapter 7 is the re-analysis chapter — AR(1) noise, hand labels, what was missing in v1. Chapter 8 documents the v2 generator. Chapters 9 to 11 report the v2 results: training, event-level evaluation, robustness sweeps, real-data inference. Chapter 12 concludes.

All code, data generation scripts and figure-generation utilities necessary to reproduce the contents of this report are included in the project repository.

Chapter 2

Theoretical background

This chapter gathers the mathematical machinery that the three architectures of chapter 5 rest on. The goal is to make the report *self-contained*: a reader without prior exposure to deep learning should, after this chapter, be able to read off the operations performed inside each of our three networks and understand why they were chosen.

We start with the abstract framing of the task as a windowed classification problem (section 2.1). We then describe in detail the three building blocks that compose our architectures: convolutional layers and the supporting normalisations (section 2.2), residual connections (section 2.3), and the multi-head self-attention mechanism that powers Transformer encoders (section 2.4). Finally, we summarise the training ingredients that all three networks share (section 2.5).

2.1 Framing: time-series classification by windowing

Let $x \in \mathbb{R}^N$ denote the gamma-dose signal sampled at uniform intervals (one minute, in our case). We want to assign each sample x_t to one of K classes: in the first iteration of the project $K = 7$ (six anomaly templates plus a **Normal Background** class), and in the second iteration the template family was pruned to $K = 4$.

Windowing. Operating on a single sample is hopeless: the classes are defined by the temporal *shape* of an event, not by the instantaneous value. We therefore process the signal through a sliding window of length W samples:

$$\mathbf{x}^{(t)} = (x_{t-W+1}, x_{t-W+2}, \dots, x_t) \in \mathbb{R}^W, \quad (2.1)$$

and we ask each network to output, from the window $\mathbf{x}^{(t)}$, a probability distribution $\hat{p}^{(t)} \in \Delta^{K-1}$ over the K classes¹. In our experiments $W = 512$ samples (about 8.5 hours at 1 sample / minute).

Successive windows overlap (consecutive predictions share most of their input), which lets us *accumulate* probabilities at inference time into a continuous, sample-level confidence trace (section 6.1.2). At training time, windows are extracted at a larger stride (32 samples) to reduce redundancy.

Labelling rule. Because a single window may contain a mixture of background and event samples, we use an *event-coverage* rule: for every event e overlapping the window, we compute the fraction of e 's own samples that the window captures, and the window inherits the class of

¹Here Δ^{K-1} denotes the standard $(K-1)$ -simplex, i.e. the set of non-negative vectors that sum to 1.

the event with the highest such fraction, provided that fraction exceeds a threshold τ_{cov} (we use $\tau_{\text{cov}} = 0.30$). Otherwise the window is labelled **Normal Background** (see fig. 5.1 in section 5.5). This makes the boundaries of events slightly fuzzy — a deliberate choice that helps the network generalise to events whose exact start and end are uncertain — while ensuring that short events (whose total duration is small compared to W) are still seen as positive examples by the network.

A natural alternative is the *window-coverage* rule: label a window positive when more than some fraction of *its own* samples are anomalous. With $W = 512$ and a threshold of 10%, this rule silently maps any event shorter than $\sim 0.1 W \approx 50$ samples to **Normal Background** regardless of how completely a window captures it, because such an event simply cannot fill enough of the window to clear the threshold. The v2 generator deliberately produces events as short as ~ 30 samples, so anchoring the threshold to the event’s own length rather than the window’s avoids losing them at the labelling stage.

Mathematical objective. Letting $f_\theta : \mathbb{R}^W \rightarrow \mathbb{R}^K$ denote the network with parameters θ , the empirical objective is the (weighted) cross-entropy

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{(\mathbf{x}, y) \in \mathcal{D}} w_y \log \text{softmax}(f_\theta(\mathbf{x}))_y, \quad (2.2)$$

where $\mathcal{D}$ is the training set of (window, label) pairs, $y \in \{0, \dots, K-1\}$ the integer label, and $w_y > 0$ a class weight that compensates for class imbalance, defined in section 2.5.2.

2.2 Convolutional layers and supporting normalisations

The cornerstone of all three architectures is the **one-dimensional convolution**.

2.2.1 The 1D convolution operator

Let $\mathbf{u} \in \mathbb{R}^{C_{\text{in}} \times L}$ denote an input sequence with C_{in} channels and length L , and let $\mathbf{w} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k}$ denote a learnable kernel of C_{out} output channels with support of size k . The convolution output $\mathbf{v} \in \mathbb{R}^{C_{\text{out}} \times L'}$ has entries:

$$\mathbf{v}_{c,t} = \sum_{c'=1}^{C_{\text{in}}} \sum_{i=1}^k \mathbf{w}_{c,c',i} \cdot \mathbf{u}_{c',s \cdot t + i - p} + b_c, \quad (2.3)$$

where b_c is a learnable bias, s is the *stride* and p is the *padding*. The output length is $L' = \lfloor (L + 2p - k)/s \rfloor + 1$. Three observations matter:

- The kernel $\mathbf{w}_{c,\cdot,\cdot}$ is shared across all time positions t , so the operator is *translation equivariant*: shifting the input shifts the output by the same amount. This is exactly what we want for an anomaly that may occur anywhere in the window.
- The number of parameters is $C_{\text{out}} \cdot C_{\text{in}} \cdot k$ (plus C_{out} biases), *independent of L* . Convolutions are therefore much cheaper than the dense fully-connected alternative, which would have $C_{\text{out}} \cdot C_{\text{in}} \cdot L \cdot L'$ parameters.
- Each output channel c acts as a learned *feature detector*: it lights up where the input matches a particular local pattern of length k .

Receptive field. Stacking several convolutional layers, optionally with sub-sampling (stride or pooling, see below), the *receptive field* of a deeper neuron — the range of input samples that can affect its value — grows roughly linearly with depth and exponentially with the sub-sampling

factor. Our $W = 512$ window is short enough that three stride-2 pooling steps (giving a receptive field of ~ 50 samples on top of $k = 7$ kernels) already cover the duration of the smallest expected anomalies (period ≥ 200 samples once temporal pooling is accounted for).

2.2.2 Pointwise nonlinearity: ReLU

Stacking purely linear convolutions yields, by composition, another linear operator and so cannot learn rich features. Between successive convolutions we therefore apply an element-wise non-linearity. We use the *rectified linear unit*, $\text{ReLU}(x) = \max(0, x)$, which is cheap to compute, has a non-vanishing gradient on its positive half-line, and has become the de facto standard activation in convolutional networks. The two normalisation variants used below already absorb the negative-half collapse problem (“dying ReLU”), so we do not need a fancier activation.

2.2.3 Pooling

After a few convolutions we typically reduce the temporal resolution by *pooling* a contiguous group of values into one. Our networks use two kinds of pooling.

Max-pooling. Of the values in a window of size k_p , keep only the maximum: $\text{MaxPool}_k(u)_t = \max_{i \in [1, k_p]} u_{s_p t + i}$, with stride $s_p = k_p$ in our case so that successive max-pools do not overlap. The output is shorter by a factor k_p . Max-pooling makes the representation locally invariant to small temporal shifts of an anomaly.

Adaptive average pooling. Used near the end of the network to bring the temporal axis down to a fixed length regardless of the input window size: it averages the input into a target number of bins. Our networks emit 4 temporal bins from `AdaptiveAvgPool1d(4)`, which preserves a coarse temporal shape (“did the activation peak in the first or the last quarter of the window?”) while removing the exact location, before the classifier head.

2.2.4 Batch normalisation

A common pathology of deep networks is that the distribution of activations inside the network shifts during training, slowing down convergence (the so-called *internal covariate shift*). Batch normalisation [5] addresses this by standardising each channel of each layer over the current mini-batch:

$$\text{BN}(\mathbf{u})_{c,t} = \gamma_c \cdot \frac{\mathbf{u}_{c,t} - \mu_c}{\sqrt{\sigma_c^2 + \varepsilon}} + \beta_c, \quad (2.4)$$

where μ_c, σ_c^2 are the channel mean and variance computed over the batch dimension (and the time dimension for 1D-BN), and γ_c, β_c are learnable rescaling parameters.

At inference time, μ_c and σ_c^2 are replaced by exponential moving averages collected during training. This means BN behaves *differently* at training and inference time, which can become problematic when there is a distribution shift between training and deployment — a subtlety that motivates the GroupNorm variant in our two more advanced architectures.

2.2.5 Group normalisation

Group normalisation [8] performs the same affine standardisation *but* computes the statistics μ, σ^2 over a partition of the channels into G groups, independently for each sample. There are no running statistics to maintain; train and inference behaviour are identical, and the operator becomes amplitude-invariant in a window-by-window fashion. This robustness against shifts in absolute scale is exactly the property we need in a stem that processes signals with a fluctuating

baseline. Our ResNet 1D and CNN+Attention models use GroupNorm with 8 groups (or fewer for narrow channel widths) throughout their convolutional stems.

2.3 Residual connections

The vanishing-gradient problem. A pure stack of convolutional layers $f_1 \circ f_2 \circ \dots \circ f_L$ becomes harder to optimise as L grows: the gradient of the loss with respect to early-layer parameters is a product of L Jacobian factors, and unless each of them is exactly orthogonal it tends to either vanish or explode exponentially.

Residual blocks. He et al. [3] proposed to reformulate a stack of two or three layers as a learnable *residual* to the identity:

$$\text{ResBlock}(\mathbf{u}) = \text{ReLU}(\mathbf{u} + \mathcal{F}(\mathbf{u})), \quad (2.5)$$

where $\mathcal{F}$ is a small sub-network (here: Conv1D $\rightarrow$ GN $\rightarrow$ ReLU $\rightarrow$ Conv1D $\rightarrow$ GN). The shortcut $+\mathbf{u}$ guarantees that the block can degenerate to the identity ($\mathcal{F} = 0$) at no cost, which means adding a residual block to a working network cannot make things worse. In gradient terms the backward pass propagates *additively* through the shortcut, so deep stacks remain trainable.

When the input and the output of $\mathcal{F}$ have different channel counts, the shortcut is replaced by a 1×1 convolution that projects them into compatible spaces. We use this only at block transitions, where the channel width grows from $16 \rightarrow 32 \rightarrow 64$.

Why residual learning helps anomaly classification. For our problem the residual block can be intuitively read as “copy the input through, then *add* a learnt correction”: in early blocks the correction extracts local shape primitives (peaks, ramps); in later blocks it refines them by re-using the original signal context that has been preserved by the shortcut.

2.4 Attention and the Transformer encoder

The third architecture in this report augments a small convolutional stem with a *Transformer encoder* [7]. Transformers are now the dominant architecture in natural language and sequence modelling; they have also proved very competitive on time-series classification [4]. The mechanism that powers them is *self-attention*, which we now describe carefully.

2.4.1 Self-attention

The input to a self-attention layer is a sequence of T token vectors $\mathbf{Z} \in \mathbb{R}^{T \times d}$ (one vector per time step, of dimension d). The layer first projects $\mathbf{Z}$ into three learnable representations called *queries*, *keys* and *values*:

$$\mathbf{Q} = \mathbf{Z}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{Z}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{Z}\mathbf{W}^V, \quad (2.6)$$

where $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V \in \mathbb{R}^{d \times d_h}$ are learnable parameter matrices (d_h is the per-head dimension).

The attention output is then

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_h}}\right) \mathbf{V}. \quad (2.7)$$

The matrix $\mathbf{A} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top/\sqrt{d_h}) \in \mathbb{R}^{T \times T}$ has rows that sum to 1; its entry A_{ij} is the fraction of token i ’s output that comes from token j . The output sequence at row i is therefore a

convex combination of all input value vectors, weighted by how compatible the query of token i is with the key of each other token. The scaling by $\sqrt{d_h}$ keeps the dot products from saturating the softmax in high dimensions.

Why this matters for our problem. Convolutional layers have a bounded receptive field: a neuron in layer ℓ depends only on samples within a window of size proportional to ℓ . Self-attention, by contrast, lets every output position attend to *every* input position in a single layer, with a *learnt*, data-dependent weighting. This long-range, content-based mixing is exactly what is needed to disambiguate, say, a sharp rise that could be a `fast_ascend` or just a noise excursion, by looking at what happens *after* the rise within the same window.

2.4.2 Multi-head attention

A single attention head can only learn one notion of similarity. To let the model attend to several relations in parallel, multi-head attention runs h independent attention heads on parallel projections of the same input, and concatenates their outputs:

$$\text{MultiHead}(\mathbf{Z}) = \text{Concat}(\mathbf{O}_1, \dots, \mathbf{O}_h) \mathbf{W}^O, \quad \mathbf{O}_j = \text{Attention}(\mathbf{Z} \mathbf{W}_j^Q, \mathbf{Z} \mathbf{W}_j^K, \mathbf{Z} \mathbf{W}_j^V). \quad (2.8)$$

We use $h = 4$ heads with $d_h = d/h = 16$, so the total number of attention parameters is the same as that of a single head with dimension $d = 64$.

2.4.3 Positional encoding

The attention operation eq. (2.7) is *permutation equivariant* in the input: shuffling the tokens of $\mathbf{Z}$ shuffles the output rows in the same way. For time series this is a problem: we explicitly want the network to know whether a feature occurred at the start or at the end of the window.

We follow Vaswani et al. [7] and add a fixed *sinusoidal positional encoding* to the token sequence before feeding it to the encoder:

$$\text{PE}_{t,2k} = \sin(t \cdot \omega^{-2k/d}), \quad \text{PE}_{t,2k+1} = \cos(t \cdot \omega^{-2k/d}), \quad \omega = 10\,000. \quad (2.9)$$

This vector is added to the token at position t before any attention layer is applied. Sines and cosines at geometrically decreasing frequencies let the network express both fine and coarse positional relations as easily-learned linear combinations.

2.4.4 The Transformer encoder block

The complete *Transformer encoder layer* composes multi-head self-attention with a two-layer feed-forward network and two LayerNorm [1] steps wrapped in residual connections (fig. 2.1):

$$\mathbf{Z}' = \mathbf{Z} + \text{MultiHead}(\text{LN}(\mathbf{Z})), \quad (2.10)$$

$$\mathbf{Z}'' = \mathbf{Z}' + \text{FFN}(\text{LN}(\mathbf{Z}')), \quad (2.11)$$

with $\text{FFN}(x) = \mathbf{W}_2 \text{GELU}(\mathbf{W}_1 x + \mathbf{b}_1) + \mathbf{b}_2$. The pre-LayerNorm convention used here (LN *before* attention/FFN) is more stable to train than the original post-LN convention.

The CLS aggregation token. After the encoder we still have a length- T sequence, but we need a single class probability. Following BERT [2], we prepend to the sequence a special learnable vector — the *classification token* [CLS] — which has no signal content of its own but is allowed to attend to every other token. After the encoder, the output row at the [CLS] position has aggregated information from the entire sequence and is fed alone to the linear classifier.

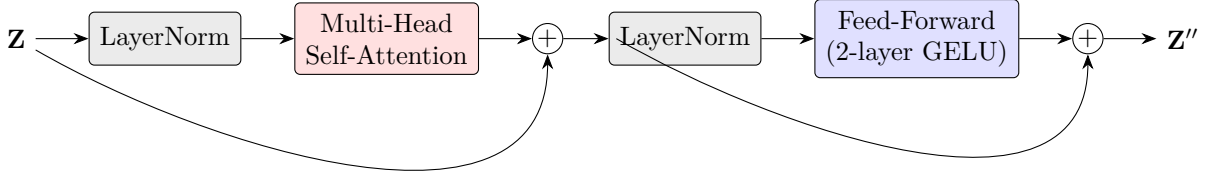


Figure 2.1. A pre-LN Transformer encoder block. Each sub-layer is wrapped in a residual connection; LayerNorm is applied before the sub-layer (pre-norm).

2.5 Training fundamentals

All three networks are trained with the same recipe.

2.5.1 Cross-entropy loss

For multi-class classification, the standard loss is the *categorical cross-entropy* introduced in eq. (2.2). Letting $\mathbf{z} = f_{\theta}(\mathbf{x}) \in \mathbb{R}^K$ be the network’s output logits and y the integer label, the contribution of a single example to the loss is

$$\mathcal{L}(\mathbf{x}, y) = -\log \frac{\exp(z_y)}{\sum_{c=1}^K \exp(z_c)}, \quad (2.12)$$

i.e. minus the logarithm of the probability the network assigns to the correct class after a softmax normalisation. The gradient of $\mathcal{L}$ with respect to z_c is $\text{softmax}(\mathbf{z})_c - \mathbf{1}[c = y]$: the network is encouraged to push the correct logit up and pull the others down, in proportion to how confident the current (wrong) belief is.

2.5.2 Class weighting

When the training set is imbalanced, the loss is dominated by the majority class and minority classes are poorly learned. We compensate with a per-class weight (a per-example multiplier on the loss) inversely proportional to the class frequency,

$$w_c = \frac{N}{K \cdot n_c}, \quad (2.13)$$

where n_c is the count of class c in the training set and N is the total count. The numerical values that this rule produces are reported, for both v1 and v2, in the dedicated training sections; they will play a non-trivial role at deployment because the training-vs-deployment class prior shift is the dominant cause of v1’s failure on real data (chapter 6).

2.5.3 Adam optimiser

The model is fit by stochastic gradient descent on mini-batches with the Adam optimiser [6], which maintains, in addition to the gradient, an exponential moving average of the first and second moments of the gradient per parameter and rescales the update accordingly:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad \theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}, \quad (2.14)$$

where g_t is the gradient at step t and $\hat{m}_t, \hat{v}_t$ are the bias-corrected moments. The defaults $\beta_1 = 0.9, \beta_2 = 0.999, \varepsilon = 10^{-8}$ are used. The initial learning rate is $\eta = 10^{-3}$.

2.5.4 Learning rate scheduling

A constant learning rate is rarely optimal: large steps early in training are useful to escape poor local minima, but late in training they oscillate around the minimum. We use `ReduceLROnPlateau`: whenever the validation loss has not improved for two consecutive epochs, the learning rate is multiplied by 0.5. This is the simplest form of adaptive schedule and is enough here to drive the loss down by roughly 20% in the last third of training.

2.5.5 Early stopping

To avoid over-fitting we monitor the validation loss and stop training when it has not improved by more than $\delta = 5 \cdot 10^{-3}$ for ten consecutive epochs. The model weights with the lowest validation loss are then re-loaded.

2.5.6 Temporal train/validation/test split

A subtle pitfall of sliding-window time-series classification is *data leakage*: if windows are extracted first and then shuffled across train/test, two windows that overlap in time can end up in different splits, so the test set effectively includes parts of the training signal. We avoid this by splitting the *raw signal* along time first (the first 80% becomes the train+validation pool, the remaining 20% becomes the test set), and only then applying the sliding-window extraction independently to each split. The 80% train+val pool is then further split uniformly at random into 80%/20%, which is safe because no overlap is possible anymore between training and validation windows on the same signal slice if the stride matches.

2.5.7 Window normalisation

Because the baseline of the signal drifts slowly (section 3.1), feeding the absolute dose to the network is inefficient: the network would waste capacity to learn the same features at every possible baseline. We *centre* each window independently — subtract its own mean — and divide by a single global standard deviation σ_{train} estimated on the centred training set:

$$\tilde{\mathbf{x}}^{(t)} = \frac{\mathbf{x}^{(t)} - \bar{\mathbf{x}}^{(t)}}{\sigma_{\text{train}}}. \quad (2.15)$$

The same σ_{train} is used at inference; saving it in a JSON file alongside the model weights guarantees deterministic and reproducible scaling between training and deployment. This window-level centring is a strong inductive bias — the network only ever sees deviations from the local baseline — and the consequences of that bias on long sustained events will surface in chapter 6.

Chapter 3

Real 2015 data and the v1 noise model

This chapter introduces the real-world data the project is anchored to (section 3.1) and the first-pass statistical model the *v1* generator was built on (section 3.2). The *v1* generator itself — templates, deformations, the end-to-end algorithm — is described in chapter 4. The deeper background re-analysis that motivated the *v2* generator is deferred to chapter 7.

3.1 Real 2015 gamma dose data

3.1.1 Source

The reference recordings are four months of gamma dose measurements collected in 2015 by an environmental surveillance beacon and provided by the project supervisor. The dates are

24/02/2015 11:20 30/04/2015 13:10 22/06/2015 07:31 20/10/2015 09:18,

each segment lasting approximately 40 000 minutes (about 28 days) at one sample per minute. The four months are stored as separate columns in a single text file under `data-gen/data/`.

Real gamma dose: four months of 2015 surveillance data



Figure 3.1. The four months of real gamma-dose recordings. Each panel is a continuous $\sim 40\text{k}$ -sample segment at one sample per minute. The signal is centred around ~ 99 counts with a high-frequency dispersion of a few counts and slow drifts of a few units; a small number of sharper excursions are visible.

3.1.2 Statistical characterisation

Figure 3.1 suggests that the signal is the superposition of (i) a slowly-varying baseline and (ii) a high-frequency, zero-mean fluctuation. We confirm this with a moving-average decomposition of window length $w = 480$ samples (fig. 3.2):

$$b_t = \frac{1}{w} \sum_{i=-w/2}^{w/2} x_{t+i}, \quad r_t = x_t - b_t. \quad (3.1)$$

Decomposition of one month of real gamma data (10k-point snapshot)

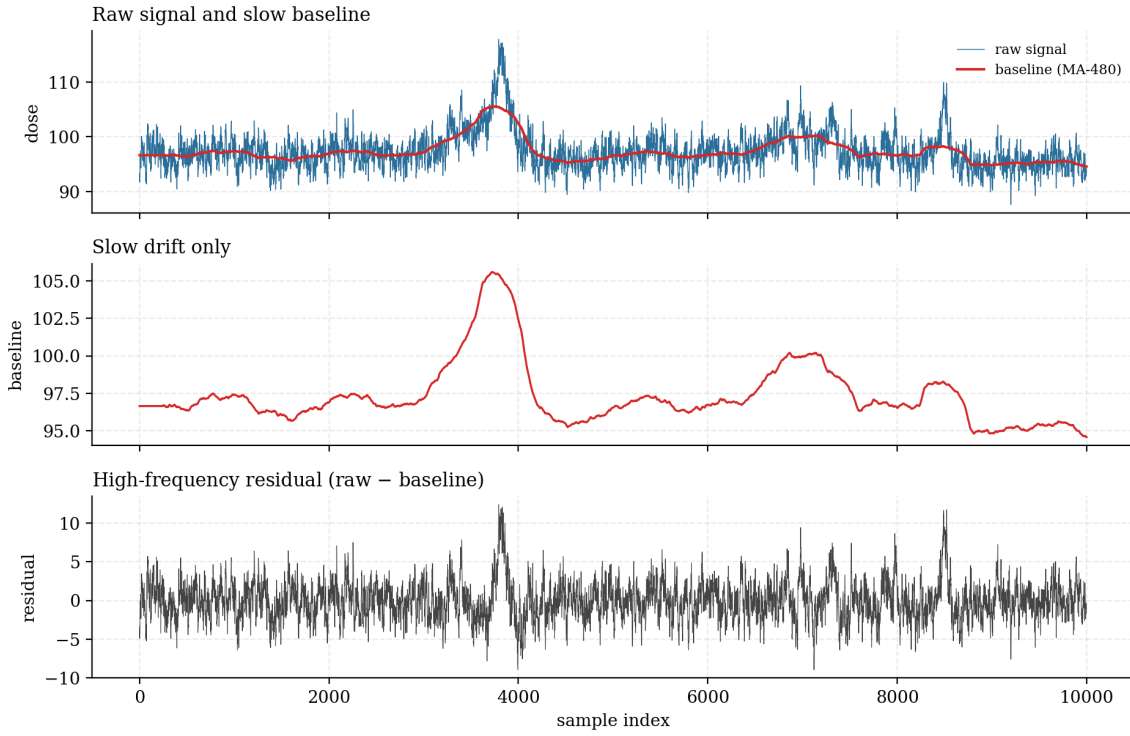


Figure 3.2. A 10 000-sample snapshot of the first month decomposed into a slowly-varying baseline (top, with the MA-480 baseline in red) and zero-mean residuals (bottom).

The empirical statistics, averaged across the four months, are reported in table 3.1. The baseline standard deviation is much smaller than the high-frequency residual standard deviation, and its slow variation is what motivates the random-walk baseline model used in the v1 generator.

Month	Global mean μ	HF noise std σ	OU θ	OU σ_{OU}
24/02/2015	96.93	2.31	$4 \cdot 10^{-6}$	0.0277
30/04/2015	98.86	2.58	$4 \cdot 10^{-6}$	0.0390
22/06/2015	101.28	2.51	$4 \cdot 10^{-6}$	0.0337
20/10/2015	99.95	2.70	$4 \cdot 10^{-6}$	0.0485
Average across all	99.26	2.53	$4 \cdot 10^{-6}$	0.0373

Table 3.1. First-pass statistical characterisation of the four months of real gamma data, as extracted by the v1 calibration script `evaluate_real_data.py`. The averaged values in the bottom row are those used in the v1 generator.

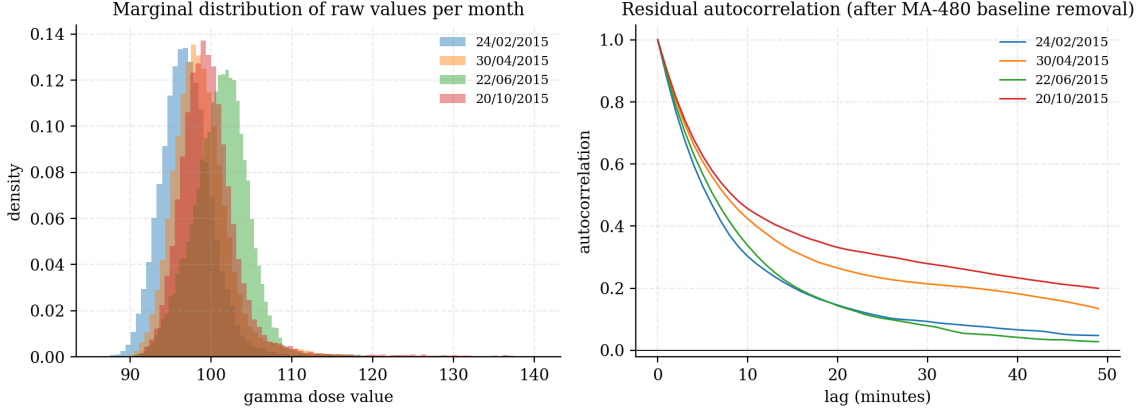


Figure 3.3. **Left:** empirical distribution of the raw gamma dose values per month. The distributions are unimodal and only slightly skewed, supporting a Gaussian approximation of the high-frequency noise. **Right:** residual autocorrelation (after MA-480 baseline removal). The non-trivial decay over ~ 30 lags is the v1 noise model’s biggest oversight; we will come back to it in chapter 7.

3.2 v1 noise model: Gaussian fluctuations on an OU baseline

Following the empirical findings of section 3.1.2, v1 models the real gamma dose as

$$x_t = b_t + \eta_t, \quad \eta_t \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2), \quad b_{t+1} = b_t + \theta(\mu - b_t) + \xi_t, \quad \xi_t \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma_{\text{OU}}^2). \quad (3.2)$$

The first equation says that the instantaneous reading is a baseline plus zero-mean Gaussian noise; the second is the Ornstein–Uhlenbeck (OU) discrete dynamics for the baseline, with mean-reversion strength θ and innovation magnitude σ_{OU} . With $\theta = 4 \cdot 10^{-6}$ the baseline is very loosely tethered — the half-life of an excursion is roughly $\log 2 / \theta \approx 1.7 \cdot 10^5$ samples, much longer than our $W = 512$ analysis windows — which reproduces the slow drifts visible in fig. 3.1.

What v1 gets right. The marginal distribution of $\eta_t \sim \mathcal{N}(0, \sigma^2)$ matches the empirical histogram of the moving-average residual (fig. 3.3, left). The OU baseline matches the order of magnitude of the long-term drift.

What v1 misses. The IID assumption for η_t is too strong: the empirical residual ACF in fig. 3.3 (right) shows a smooth decay over ~ 30 lags rather than a sharp drop to the $\pm 2/\sqrt{N}$ band that white noise would produce. At the level of the marginal histogram and the global standard deviation, the mismatch is invisible; at the level of the local texture of a 512-sample window — the input that a convolutional stem actually sees — it is large. Chapter 7 will fix this with an AR(1) model.

Chapter 4

The v1 synthetic generator

This chapter documents the first iteration of the synthetic data generator. It is kept deliberately compact because the v1 generator is no longer the recommended pipeline; its main role in the rest of the report is as a baseline that the v2 generator (chapter 8) improves upon. The reader interested in the final pipeline can skim this chapter and jump straight to chapters 7 and 8.

4.1 Anomaly templates (v1)

The six v1 anomaly templates are stored as small CSV files in `pole-anomaly-old/data-gen/anomalies/`. Each row defines a *keypoint* with six fields plus an interpolation flag; the schema is documented in section A.1. Visually, the templates fall into four families (fig. 4.1):

- **Smooth bumps** (`bell`, `bell_sq`): plumes that build up and decay smoothly, possibly with a small sustained plateau.
- **Asymmetric ramps** (`fast_ascend`, `fast_descend`): rapid onset followed by slow exponential decay, or its time-reverse.
- **Plateau** (`square`): a sustained contamination episode.
- **Multi-peak** (`M`): two consecutive events with a dip in between.

v1 generator: six hand-designed anomaly templates

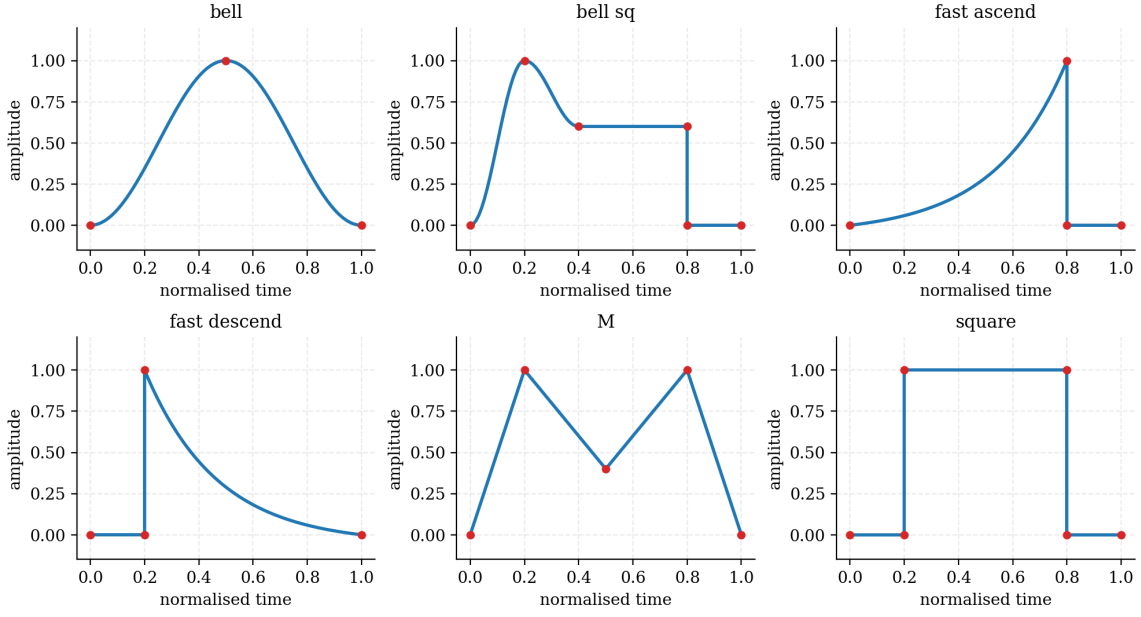


Figure 4.1. The six v1 anomaly templates, drawn in normalised time and amplitude. Each template is encoded as a small list of keypoints (red dots) connected by linear, exponential or half-cosine (“bell”) segments.

4.1.1 Stochastic deformation of a template

A template used verbatim would not generalise. The generator therefore *deforms* each template before injecting it: each keypoint is perturbed by an isotropic Gaussian jitter of standard deviation ν (the *variance level*), then the direction constraints encoded by the `t_minus`, ..., `v_plus` flags are enforced so that the deformed template still has the right qualitative shape (a peak cannot become a valley, time monotonicity is enforced as a fail-safe). The deformed keypoints are then interpolated back into a dense curve and re-scaled to the target amplitude and period; v1 uses

$$\text{amplitude} \in \sigma \cdot [2.5, 7.6], \quad \text{period} \in [200, 800] \text{ samples}, \quad \nu \in [0.02, 0.10]. \quad (4.1)$$

The amplitude is expressed in multiples of the high-frequency noise standard deviation σ from table 3.1, so the per-sample SNR of a generated anomaly lies in the interesting range $[2.5, 7.6]$.

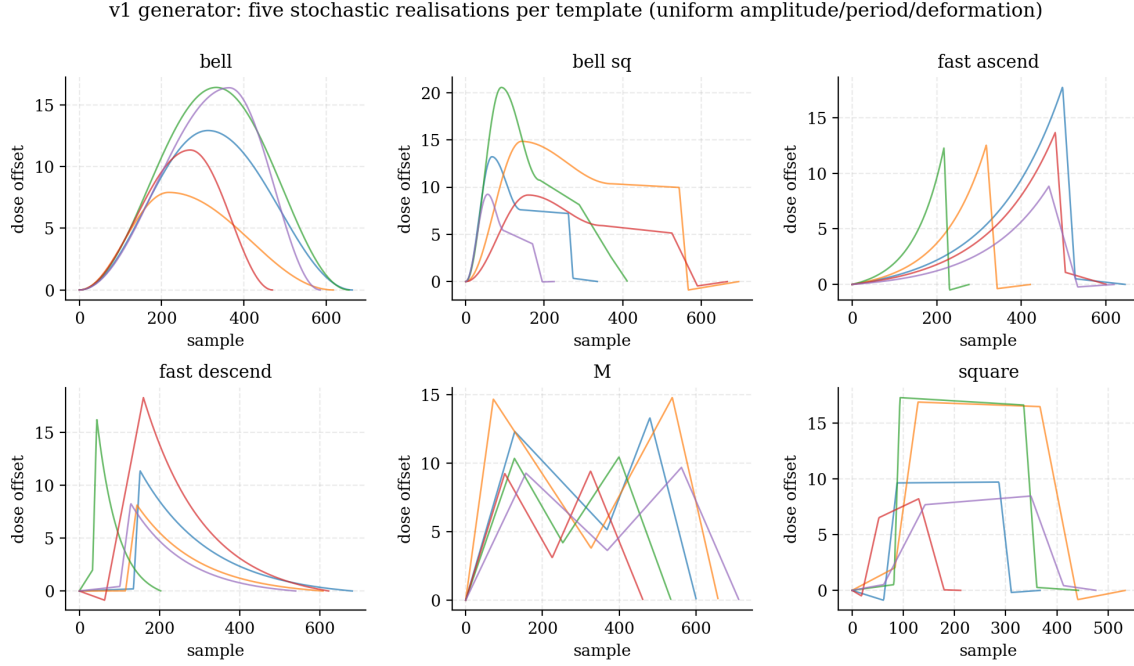


Figure 4.2. Five stochastic realisations of each v1 template with the parameter ranges of eq. (4.1). The qualitative shape is preserved, but no two realisations are identical.

4.2 The v1 generator algorithm

The end-to-end generator (`generate_custom_dataset.py` in `pole-anomaly-old/data-gen/`) combines the noise model of section 3.2 and the deformed templates of section 4.1.1 into a long labelled signal. In pseudo-code, generating N points proceeds as:

1. Pick the total number of anomalies $M \in [5\,000, 8\,000]$ and pre-compute their evenly-spaced injection indices, with a small random jitter (± 200 samples) around each grid point.
2. For each injection point, sample a template uniformly at random and a triple (amplitude, period, ν) from the ranges of eq. (4.1); produce the deformed dense curve.
3. Generate the background in 50 000-point chunks: simulate the OU baseline path and overlay $\mathcal{N}(b_t, \sigma^2)$ noise.
4. Add each anomaly curve on top of the background at its injection index and label every output point with the anomaly's class (or 0 if the per-point contribution falls below the threshold).

The final dataset is written to disk in CSV chunks; it never lives entirely in RAM, which lets us produce datasets of arbitrary size within a constant memory budget.

4.3 The v1 dataset used in this report

For the v1 results in chapter 6 we generate $N = 10^7$ samples and inject in the range $[5\,000, 8\,000]$ anomalies, leading to a global anomaly density of ~ 7 events per 10 000 samples. The signal looks as in fig. 4.3.

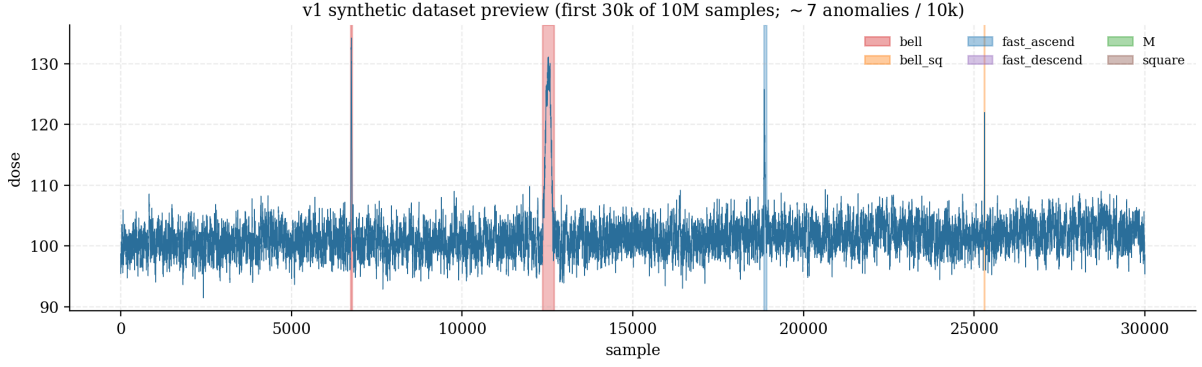


Figure 4.3. First 30 000 samples (of 10^7) of the v1 synthetic dataset. Anomaly spans are coloured by class. Note the dense injection: roughly four visible bumps in a 30k-sample slice.

4.4 Two v1 design choices that will matter later

Two v1 choices, harmless on the synthetic test set, turn out to be the main causes of the failure on real data (chapter 6):

Dense injection. The synthetic dataset contains roughly one anomaly every 1 500 samples, whereas the real recordings contain 17 plausible anomalies over $4 \times 40\,000$ samples (section 7.5). The two distributions are off by a factor of ~ 70 in event density — and so the v1 training prior of `Normal Background` is wildly off the deployment prior.

Templates assume clean, isolated, well-defined shapes. The v1 generator produces anomalies that are isolated and have a duration in $[200, 800]$ samples. Real-life events may be much shorter (a few samples) or much longer (multi-hour contamination), and the recordings contain rapid, isolated single-point excursions that the templates do not represent at all.

The next two chapters first describe the three architectures shared by v1 and v2 (chapter 5), then report the v1 training and event-level evaluation on the synthetic test set (chapter 6) and the v1 real-data inference (chapter 6). Readers familiar with the v1 results can skim them; the takeaway is that v1 works very well on synthetic data and very badly on real data.

Chapter 5

Architectures and training pipeline

This chapter is the engineering counterpart of chapter 2. Sections 5.1 to 5.3 describe the three neural networks we trained, with their exact layer composition and the intent behind each design choice. Section 5.4 summarises the three in a single comparison table. Section 5.5 then documents the shared training pipeline.

The three architectures are *identical* between v1 and v2 — only the input dataset and the number of classes change. The class count is set by the data and is the only architecture knob that depends on it: $K = 7$ for v1, $K = 4$ for v2. All structural parameters (channel widths, kernel sizes, attention heads, dropout) are the same in both iterations.

The three models share the same *interface*: input $\mathbf{x} \in \mathbb{R}^{1 \times 512}$ (a single channel, a window of 512 samples), output $\hat{p} \in \mathbb{R}^K$ (probabilities over the K classes). This makes them drop-in interchangeable for the training and inference scripts, which select a model by name and otherwise behave identically.

5.1 Architecture A — baseline 1D-CNN

The first model, defined in `architectures/saved_models/1d_cnn.py`, is a deliberately compact convolutional network. It serves both as a baseline to beat and as a sanity check that the data and the training pipeline are correctly wired.

Structure. The network is a linear stack of three convolution–BatchNorm–ReLU–MaxPool blocks, followed by an adaptive average pool and a small classifier head:

Block	Operation	Output shape
Input	Raw window	$(1, 512)$
Block 1	$\text{Conv1D}(1 \rightarrow 16, k=7, p=3) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2)$	$(16, 256)$
Block 2	$\text{Conv1D}(16 \rightarrow 32, k=7, p=3) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2)$	$(32, 128)$
Block 3	$\text{Conv1D}(32 \rightarrow 64, k=5, p=2) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2)$	$(64, 64)$
Pool	$\text{AdaptiveAvgPool1d}(4)$	$(64, 4)$
Classifier	$\text{Flatten} \rightarrow \text{Dropout}(0.4) \rightarrow \text{Linear}(256 \rightarrow 32) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.2) \rightarrow \text{Linear}(32 \rightarrow K)$	(K)

Design notes.

- Kernel sizes $k = 7, 7, 5$ are large enough to capture multi-sample shape primitives at each scale while staying well within the receptive-field budget.

- Channel widths $16 \rightarrow 32 \rightarrow 64$ double at each block, compensating the temporal compression by a factor of 2 at every `MaxPool`.
- `AdaptiveAvgPool1d(4)` keeps four temporal bins after the convolutional trunk: this preserves a coarse notion of “where in the window the activation peaks”, useful for shapes such as `fast_ascend` vs `fast_descend` that differ mainly by the location of their peak.
- Two dropout layers ($p = 0.4$ then $p = 0.2$) in the fully-connected head act as a regulariser.

Total trainable parameters: 22 727 (v1, $K = 7$) / 22 628 (v2, $K = 4$). The classifier head’s output width is the only thing that changes between the two iterations.

5.2 Architecture B — 1D Residual Network

The second model, defined in `architectures/saved_models/resNet.py`, replaces each convolutional block of model A with a small *residual block* (section 2.3) and substitutes every `BatchNorm` with `GroupNorm` (section 2.2.5).

Residual block. A single block transforms an input of channel count c_{in} into an output of channel count c_{out} as

$$\mathcal{F}(\mathbf{u}) = \text{GN} \circ \text{Conv1D}_{k_3} \circ \text{ReLU} \circ \text{GN} \circ \text{Conv1D}_{k_2} \circ \text{ReLU} \circ \text{GN} \circ \text{Conv1D}_{k_1}(\mathbf{u}),$$

$$\text{Block}(\mathbf{u}) = \text{ReLU}(\mathcal{F}(\mathbf{u}) + \text{Shortcut}(\mathbf{u})),$$

where the shortcut is either the identity (when $c_{\text{in}} = c_{\text{out}}$) or a 1×1 convolution followed by `GroupNorm`. Kernel sizes (k_1, k_2, k_3) are $(15, 9, 5)$ in the first block (broad context immediately) and $(7, 5, 3)$ in the next two.

Block	Operation	Output shape
Input	Raw window	$(1, 512)$
Block 1	$\text{Res}(1 \rightarrow 16, k=(15, 9, 5))$	$(16, 512)$
Pool 1	<code>MaxPool(2)</code>	$(16, 256)$
Block 2	$\text{Res}(16 \rightarrow 32, k=(7, 5, 3))$	$(32, 256)$
Pool 2	<code>MaxPool(2)</code>	$(32, 128)$
Block 3	$\text{Res}(32 \rightarrow 64, k=(7, 5, 3))$	$(64, 128)$
Pool	<code>AdaptiveAvgPool1d(4)</code>	$(64, 4)$
Classifier	Same head as model A	(K)

Full structure.

Design notes.

- `GroupNorm` with $\min(8, c_{\text{out}})$ groups everywhere means train-time and inference-time behaviour are identical, which matters when deploying on real data with a different value distribution.
- The shortcut connections preserve the original signal through the convolutional trunk, allowing later blocks to focus on *adding* discriminative information.
- The classifier head is intentionally identical to that of model A so that the residual trunk is the only difference between the two networks — a clean ablation.

Total trainable parameters: 74 631 (v1) / 74 532 (v2).

5.3 Architecture C — CNN+Attention

The third model, defined in `architectures/saved_models/cnn_attention.py`, hybridises a convolutional stem with a Transformer encoder (section 2.4.4). The motivation is to combine the *locality* of convolutions, well-suited to short shape primitives, with the *global mixing* of attention, which lets the network reason about how those primitives compose over the whole window.

Convolutional stem. Three Conv–GroupNorm–ReLU blocks reduce the temporal length by a factor of 8 via stride-2 convolutions:

Layer	Operation	Output shape
Input	Raw window	(1, 512)
Stem 1	Conv1D(1→16, $k=7$, $s=2$, $p=3$) → GN(8) → ReLU	(16, 256)
Stem 2	Conv1D(16→32, $k=5$, $s=2$, $p=2$) → GN(8) → ReLU	(32, 128)
Stem 3	Conv1D(32→64, $k=3$, $s=2$, $p=1$) → GN(8) → ReLU	(64, 64)

Tokenisation and Transformer encoder. The stem output is reshaped to a sequence of $T = 64$ tokens of dimension $d = 64$. A learnable [CLS] token is prepended ($T' = 65$ tokens total), a sinusoidal positional encoding (eq. (2.9)) is added, and two pre-LayerNorm encoder blocks are applied. Each encoder block has $h = 4$ attention heads of dimension $d_h = 16$ and a feed-forward network with hidden size $d_{\text{ff}} = 4d = 256$, GELU activation and dropout $p = 0.1$.

Classification. The encoder output at the [CLS] position is LayerNorm-normalised and passed through a single linear layer $\mathbb{R}^d \rightarrow \mathbb{R}^K$ to produce the class logits.

Design notes.

- The stride-2 convolutional stem reduces the sequence length eightfold before attention, keeping the $O(T^2)$ cost of attention manageable at $65^2 \approx 4\,000$ pairs.
- GroupNorm (not BatchNorm) in the stem makes the entire feature extraction amplitude-invariant on a per-window basis — consistent with the window-mean centring of eq. (2.15).
- The [CLS] token is initialised with a small random vector (truncated normal, $\sigma = 0.02$) and is the only *learnable* positional information; the sinusoidal encoding is shared and fixed.

Total trainable parameters: 109 655 (v1) / 109 460 (v2).

5.4 Side-by-side comparison

Table 5.1 summarises the three architectures.

	1D-CNN	ResNet 1D	CNN+Attention
Convolutional trunk	3 plain blocks	3 residual blocks	3-layer stride-2 stem
Normalisation	BatchNorm	GroupNorm(8)	GroupNorm(8)
Skip connections	—	yes	yes (inside Transformer)
Long-range mixing	—	—	Multi-head self-attention
Output head	MLP(256 → 32 → K)	MLP(256 → 32 → K)	Linear(64 → K) on CLS
Trainable parameters (v2)	22 628	74 532	109 460

Table 5.1. Side-by-side summary of the three architectures.

5.5 Training pipeline

The training pipeline is implemented once, in `architectures/train.py`, and parameterised by the model name.

5.5.1 Data preparation

The raw signal is split temporally (80/20) into a train+val pool and a test pool (section 2.5.6), then converted into sliding windows of size 512 with stride 32. The temporal split before windowing is what protects against data leakage between train and test windows.

Windowing. Each window is centred individually (subtract its own mean) and scaled by the global training standard deviation σ_{train} , which is saved in a JSON file next to the model weights so that inference uses exactly the same scaling.

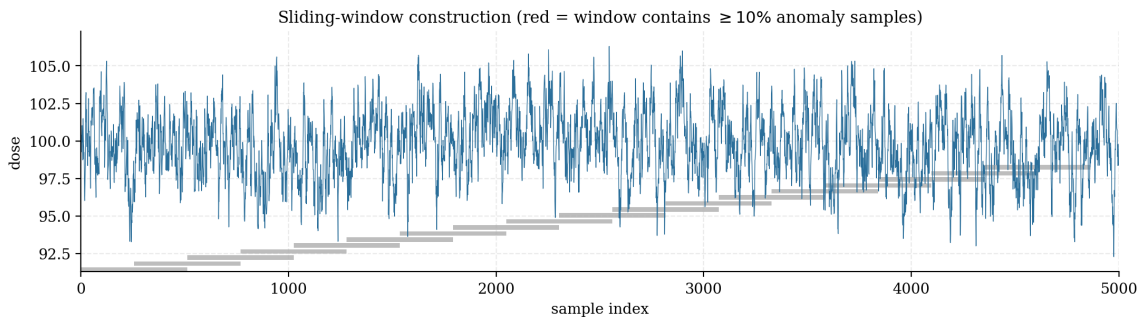


Figure 5.1. Sliding-window construction on a fragment of the v2 synthetic signal. Each coloured bar represents one 512-sample training window; red bars indicate windows that capture $\geq 30\%$ of an underlying event’s samples and so inherit its class label (event-coverage rule, section 2.1).

5.5.2 Optimisation

All three models are trained with the recipe of section 2.5: Adam ($\eta = 10^{-3}$, default β_1, β_2), batch size 128, weighted cross-entropy eq. (2.2), a `ReduceLROnPlateau` scheduler with patience 2 and factor 0.5, early stopping with patience 10 and minimum delta $5 \cdot 10^{-3}$, maximum 35 epochs. No data augmentation is applied — the stochastic anomaly generation already provides enough variability.

5.5.3 Class-weighting in v1 vs v2

The two iterations differ markedly in their training class distribution, and therefore in the class-weight vector that the loss uses (fig. 5.2).

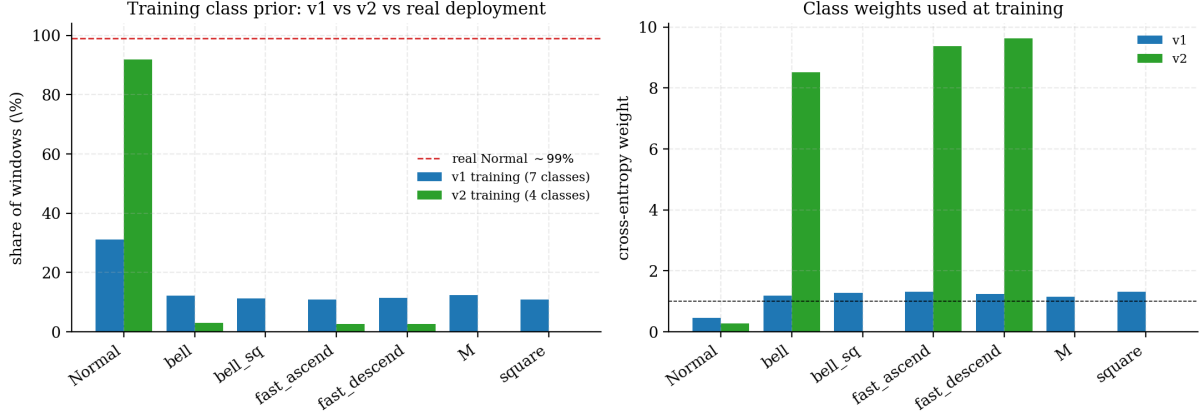


Figure 5.2. **Left:** window-level class distribution at training, for v1 (seven classes, blue) and v2 (four classes, green), against the (estimated) real deployment prior in red ($\sim 99\%$ **Normal**). v1 has been trained with a prior that is roughly the *opposite* of the deployment prior. v2 closes the gap to within a factor of three. **Right:** cross-entropy class weights used in each iteration. v2 strongly down-weights **Normal** ($w_0 = 0.27$ vs ~ 9 on the anomaly classes) because the class imbalance in v2 is also much larger than in v1.

v1 class weights. v1 has $\sim 31\%$ **Normal** and $\sim 11\%$ per anomaly class, yielding weights $\mathbf{w}_{v1} = (0.46, 1.18, 1.27, 1.32, 1.24, 1.15, 1.31)$.

v2 class weights. v2 has $\sim 92\%$ **Normal** and $\sim 3\%$ per anomaly class, yielding weights $\mathbf{w}_{v2} = (0.27, 8.51, 9.37, 9.63)$.

5.5.4 Selection criterion

The training script saves the weights with the lowest validation loss. Validation F1 score (macro-averaged across the classes) is tracked at every epoch for diagnostic purposes but is not used for selection.

Chapter 6

v1: synthetic results and real-data failure

This chapter is the v1 retrospective: how the three architectures behaved when trained on the v1 dataset, both on the held-out synthetic test segment (section 6.1) and on the four months of real recordings (section 6.3). The takeaway — the system passes the synthetic benchmark very comfortably but fails on real data — is what motivates the rest of the report.

6.1 Synthetic-test performance

6.1.1 Training dynamics

Figure 6.1 overlays the train and validation losses and the validation macro F1 of the three models.

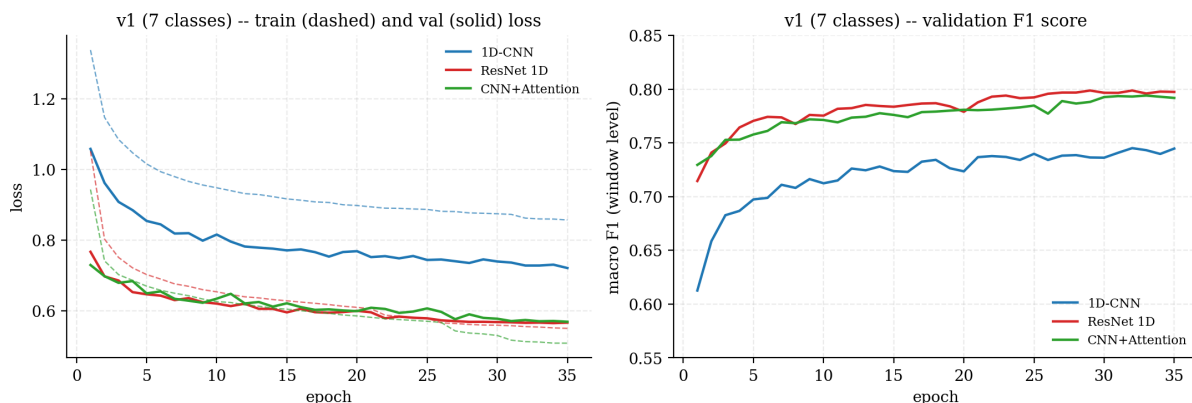


Figure 6.1. v1 training dynamics. Left: train (dashed) and validation (solid) loss for the three models. Right: validation macro F1 (window level). All three models converge within 35 epochs and benefit from one or two learning-rate reductions.

The residual network reaches a clearly lower validation-loss plateau than the baseline 1D-CNN (about 0.57 vs 0.72), and so does the attention model. Both more advanced models also start much higher on the validation F1 axis already at the first epoch (> 0.71 versus 0.61 for the baseline).

6.1.2 Event-level evaluation methodology

For an operationally meaningful metric, we evaluate the system on a continuous test segment using a *detection-driven* protocol implemented in `architectures/inference_engine.py`.

A test segment of 100 000 samples (immediately following the training portion) is fed to the network through a sliding window of size 512 and stride 10; per-window probability outputs are averaged into a per-sample probability trace $\hat{P} \in \mathbb{R}^{N \times K}$. A sample is declared anomalous if $\hat{P}_{t,0} < 0.3$; contiguous anomalous regions form predicted events, with small gaps of up to 50 samples bridged. The class of a predicted event is the *argmax* of the cumulative probability vector across its samples. True events are extracted from the ground-truth labels the same way; a predicted event matches a true event whenever the two intervals overlap, and the result is split into TPs, misclassifications, missed events, and hallucinations. Per-class precision/recall/F1 are then computed, and the report quotes the macro average across the anomaly classes.

6.1.3 v1 event-level results

On the 100 000-sample v1 test segment, there are 59 ground-truth events. The event-level confusion matrices are shown in fig. 6.2.

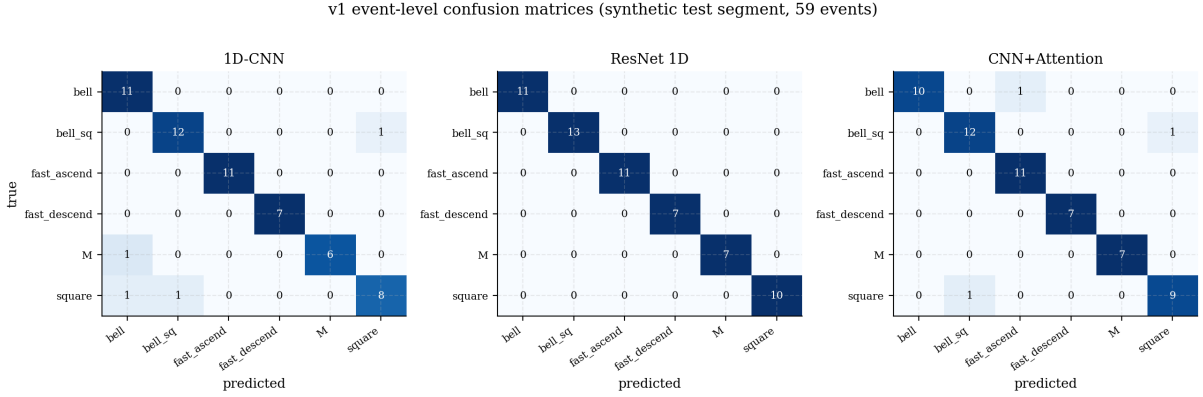


Figure 6.2. v1 event-level confusion matrices on the synthetic test segment. Cell colour is the row-normalised recall; the integer inside each cell is the raw event count. The residual network achieves a perfect classification; the attention model misses three events; the baseline 1D-CNN misses four events.

The corresponding macro F1 scores are summarised in table 6.1.

Model	Event-level macro F1
1D-CNN	0.934
ResNet 1D	1.000
CNN+Attention	0.955

Table 6.1. v1 event-level macro F1 on the synthetic test segment (59 events). All three models cross the 90% line; the residual network is flawless.

A striking property of these results is that the false-positive rate is essentially zero: predicted events that do not overlap any true event are extremely rare on this segment. The trained system looks, on its own training distribution, in very good shape.

6.2 v1 robustness study

To check whether the same hold under controlled stress, the robustness script (`architectures/robustness_test`) generates small test datasets while sweeping two distortion parameters independently: the background-noise standard deviation σ (multiplied by $m_{\text{noise}} \in [1, 3]$, with the anomaly amplitudes divided by the same factor so the absolute SNR remains constant), and the deformation-variance parameter ν (multiplied by $m_{\text{deform}} \in [1, 8]$).

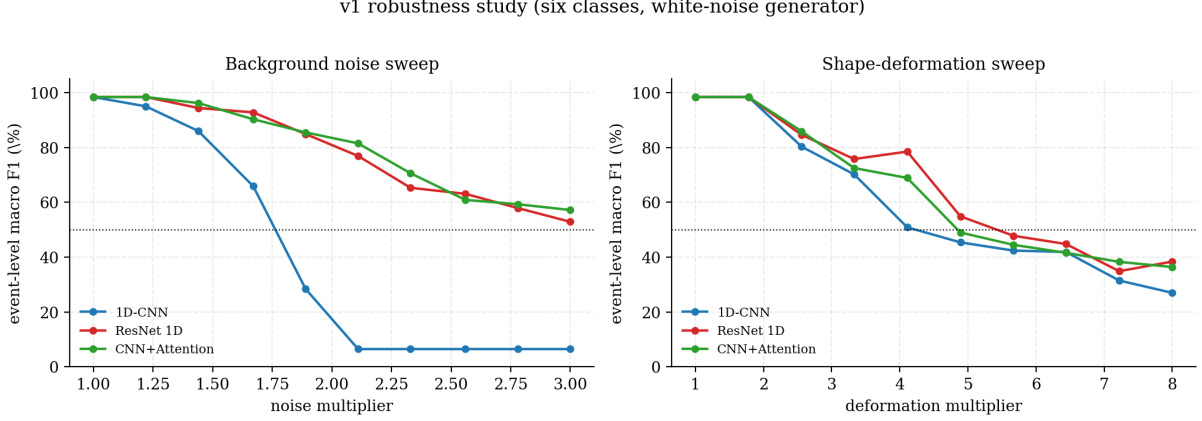


Figure 6.3. v1 robustness curves. Left: macro F1 vs. background-noise multiplier. Right: macro F1 vs. anomaly-shape deformation multiplier. The dotted line marks 50% F1. The 1D-CNN drops sharply to a $\sim 7\%$ plateau under noise — the hallmark of a classifier that has lost confidence in its features and collapses to a single class. The residual and attention models degrade much more gracefully.

The two structural takeaways from fig. 6.3 are: (i) residual connections + GroupNorm dramatically improve the noise robustness of the network (the gap between the 1D-CNN and the ResNet at $m_{\text{noise}} = 2$ is enormous); (ii) self-attention does not buy any extra robustness on top of residual learning at $W = 512$, because the window is too short for the attention mechanism to have much extra context to exploit. The full discussion is identical to what section 10.4 reports for v2.

6.3 v1 on real data: three architecture-specific failures

The clean-test numbers above hide an alarming truth: when the same trained v1 models are confronted with the real 2015 recordings, all three behave poorly — and each fails in its own way (fig. 6.4).

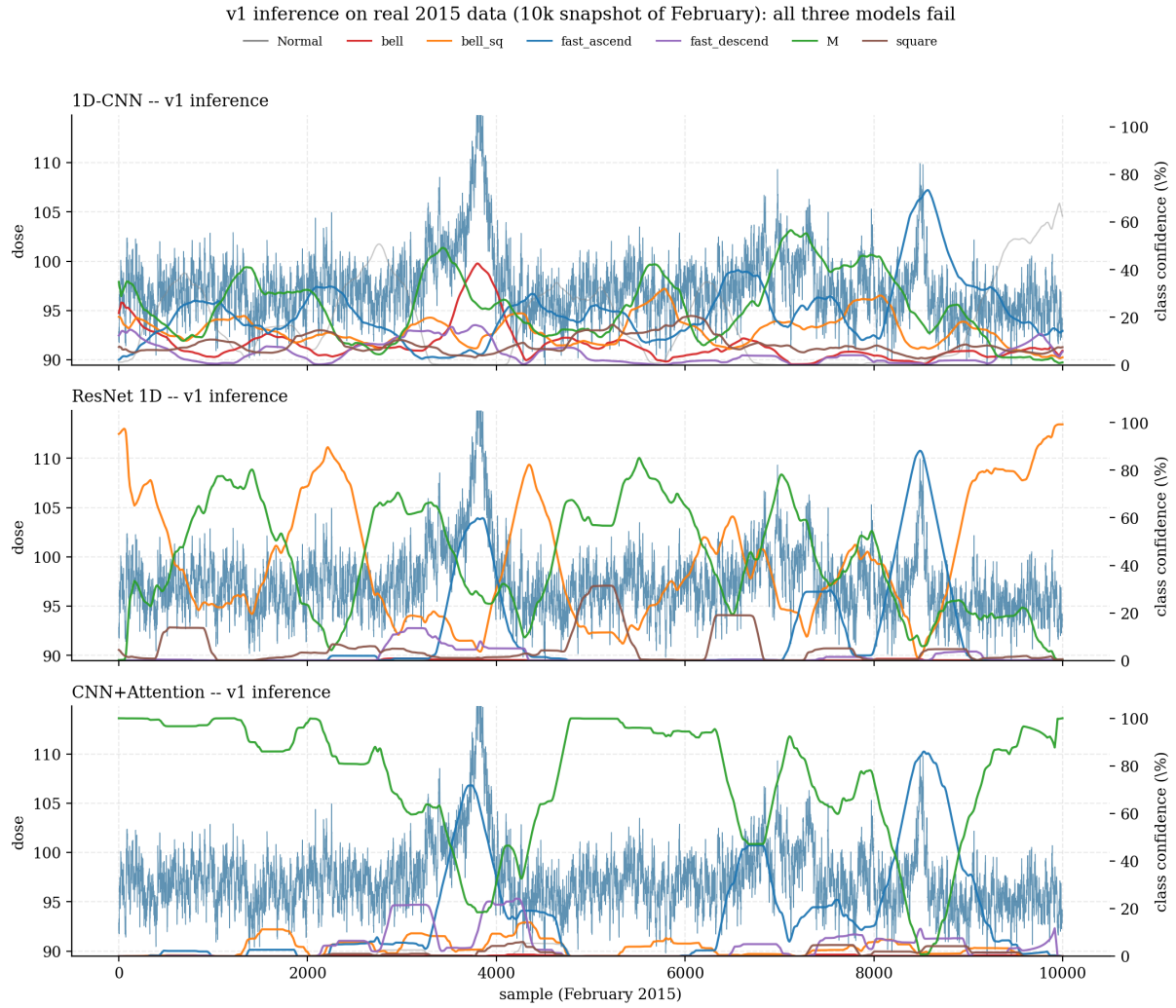


Figure 6.4. v1 inference on the first 10 000 samples of February 2015. Blue: raw gamma dose (left axis). Coloured lines: per-class confidence smoothed with a 30-sample rolling mean (right axis, 0–100%). The grey curve is the **Normal Background** confidence; the other colours are the six anomaly classes.

1D-CNN — high-frequency oscillating predictions. Each anomaly class takes its turn going up and down between 0% and $\sim 40\%$. The **Normal Background** probability sits in the 50%–70% band but never crosses the operational threshold that would make the system silent. A spike in M confidence around sample 4 000 coincides with the visible bump in the raw signal — that part is encouraging — but the same level of confidence is reached elsewhere, in places where nothing is happening.

ResNet — sustained false-positive plateaus. The ResNet does not oscillate; instead, it locks onto one anomaly class at a time for long stretches. The orange **bell_sq** confidence plateaus near 100% for thousands of samples; the green M confidence does the same on the second half of the window. Every quiet region of the signal is silently classified as one anomaly or another.

CNN+Attention — degenerate collapse onto M. The hybrid model exhibits the most dramatic failure: the green M confidence is pinned to 100% for the entire 10 000-sample snapshot. The signal could be flat at the global mean, and the model would still declare M with full confidence.

The three failures share a single root cause — a much larger *distribution shift* between training and deployment than the v1 noise model accounted for — but they are amplified differently by each architecture. We now look at the main mismatches.

6.4 Five causes of the v1 failure

Cause 1: training-vs-deployment class prior. The v1 synthetic dataset has roughly 31% **Normal** windows and $\sim 11\%$ of each anomaly class. The real recordings have $> 99\%$ **Normal** (only 17 plausible events in $4 \times 40\,000$ samples). The two priors are roughly *opposite*. On top of this, v1’s class-weight vector $\mathbf{w}_{v1}$ down-weights **Normal** by a factor ~ 2.7 relative to the anomaly classes, which from a Bayesian perspective is equivalent to multiplying the network’s posterior on the anomaly classes by another factor of ~ 2.7 at deployment. The deployment posterior is therefore mis-calibrated by a factor of roughly 200.

Cause 2: anomaly-duration mismatch. The v1 templates have durations in $[200, 800]$ samples. The hand-labelled real events span 32 to 1014 samples (section 7.5), with several events of ~ 50 samples that fall well outside the v1 range.

Cause 3: noise-texture mismatch. The v1 background is IID Gaussian. The real residual is autocorrelated with lag-1 ACF ≈ 0.86 (chapter 7). A 512-sample window therefore sees a very different local texture in synthetic vs. real — much “rougher” in v1 because the samples are independent, much smoother in the real signal because of the AR(1) memory. The marginal histogram of values inside the window is identical in both cases ($\sigma \approx 2.3$), so any model that looks only at the marginal sees no shift, but a 1D convolutional layer extracting features from local patches of a few samples sees a fundamentally different input.

Cause 4: side-effects of per-window centring. The window mean is subtracted before every forward pass (eq. (2.15)). On the v1 synthetic test set this is harmless because the events occupy a known fraction of the window. On a real long contamination plateau that covers most of the window, the window mean is dragged up by the event itself; after centring, the event has *lost most of its amplitude* and only its edges survive.

Cause 5: no “background-but-not-Normal” class. A v1 window either contains a clean templated event or pure stationary background. Real recordings contain a much richer family of intermediate signals (a slow drift, an outlier sample, a brief sensor wobble) that are neither a clean event nor pure stationary noise. With no class to assign to those, and (because of cause 1) a bias towards the anomaly classes, the network is forced into a wrong choice on every quiet region.

Why the failure mode depends on the architecture. The five causes are shared; the failure mode is not.

- **1D-CNN with BatchNorm.** BatchNorm running statistics accumulated during training are wrong at deployment, so the normalised activations are wildly different from anything the network saw at training; the classifier head can no longer commit to a single class and oscillates.
- **ResNet with GroupNorm.** GroupNorm is per-window, so normalisation behaves identically at training and deployment — the activations are valid, the features are stable — but the

classifier head is still mis-calibrated by the prior shift, so it coherently and confidently picks the wrong class.

- **CNN+Attention.** The attention layer aggregates the entire window into a single CLS token; when the window is out-of-distribution, the CLS aggregation converges to a single mode (the most “textured” class, M) regardless of the input.

6.5 What to do about it

The five causes translate into a concrete remediation list, which is exactly what motivates the v2 generator of chapter 8:

1. **Match the deployment prior.** Lower the synthetic anomaly density by an order of magnitude (cause 1).
2. **Match the noise texture.** Replace the IID Gaussian residual by an AR(1) Gaussian fitted to the real data (cause 3).
3. **Match the anomaly amplitude and duration ranges.** Drive both from the hand-labelled real events; in particular, drop the artificial duration floor at 200 samples and extend the range down to ~ 30 (cause 2).
4. **Drop the templates that do not occur in the real labels.** The M, bell_sq, square templates have no counterpart in the hand labels and are exactly the classes the v1 networks collapsed onto on out-of-distribution input (cause 5).
5. **Hand-label a small set of real events** to drive the above three calibrations (and to detect, after re-training, whether the corrections worked).

Causes 1–3 are addressed by the v2 generator (chapters 7 and 8). Cause 4 (per-window centring) is not changed in v2 — the empirical durations in the labels stay shorter than 512 samples on most events, so the per-window centring remains a reasonable inductive bias. Cause 5 is partially addressed by dropping the unused templates, but a dedicated “unknown but not Normal” class is left for future work.

The next chapter walks through the re-analysis of the real background that produces, in particular, the AR(1) noise model used in v2.

Chapter 7

Re-analysis: AR(1) noise and hand labels

The v1 failure analysis (section 6.4) identifies five distinct sources of distribution shift between the v1 synthetic training distribution and the real 2015 recordings. Before re-implementing the generator, this chapter goes back to the four months of real data and asks two questions the original v1 calibration did not pose:

1. How good a description of the background process is the IID-Gaussian-on-OU model of eq. (3.2)? Where it fails, what is a better model? (sections 7.2 to 7.4.)
2. What do the real anomalies that we can plausibly identify on the recordings actually look like — how many, how long, what amplitude, what shape? (section 7.5.)

Both lead to concrete numerical targets for the v2 generator (section 7.6).

The analysis is split across two scripts: `data-gen/analyze_noise_model.py` (AR(1) fitting, AR order selection, whitening, drift, stationarity — outputs in `data-gen/logs/noise_analysis/`) and `data-gen/labeling_view.py` (visual labelling helper — outputs in `data-gen/logs/labeling/`).

7.1 Methodology

The v1 calibration estimated the residual standard deviation as the sample standard deviation of the moving-average residual over the whole month. Two refinements are needed before the inferred statistics can be trusted.

Robust outlier masking. The signal contains visible excursions whose amplitudes inflate the sample standard deviation and distort the empirical autocorrelation. We use a rolling Median Absolute Deviation (MAD) for the local dispersion estimate (breakdown point 50%) and mask samples whose robust z -score exceeds 4 before computing residual statistics. The robust z -score is

$$z_t = \frac{x_t - b_t}{1.4826 \cdot \text{MAD}_w(x - b)_t}, \quad (7.1)$$

with b_t the centred MA-480 baseline and $1.4826 = 1/\Phi^{-1}(0.75)$ the standard scaling factor that makes the MAD a consistent estimator of the standard deviation under Gaussian assumptions. After masking, ~ 92 – 95% of each month survives as the *clean residual*, on which all noise statistics below are computed.

Multi-scale baseline. The single $w = 480$ moving average absorbs features whose duration is comparable to w . We use $w_s = 480$ for the high-frequency residual analysis and an ultra-wide $w_w = 17\,280$ baseline (*i.e.* ~ 12 days) for the drift analysis of section 7.4.

7.2 High-frequency noise is AR(1), not white

Figure 7.1 reports the diagnostic plot computed on the masked residual of February 2015.

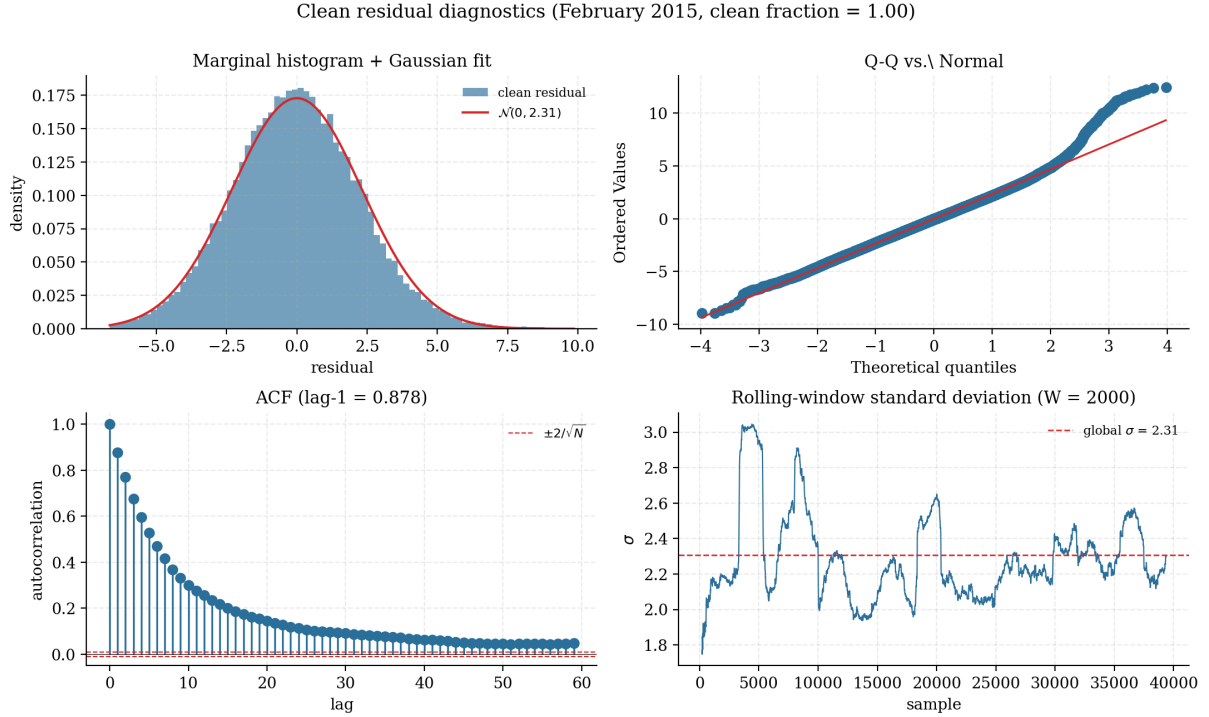


Figure 7.1. Diagnostics on the masked residual of February 2015. **Top-left:** marginal histogram, near-perfectly fit by $\mathcal{N}(0, 2.21)$. **Top-right:** Q-Q plot vs. Normal, on the diagonal. **Bottom-left:** autocorrelation function on the clean segment — $\text{ACF}(1) = 0.864$, decaying smoothly to zero by lag ~ 30 . **Bottom-right:** rolling standard deviation, stationary around the global value.

The marginal histogram and the Q-Q plot are essentially perfect Gaussian fits: the marginal distribution of the clean residual is, to a very good approximation, $\mathcal{N}(0, \sigma^2)$. That part of the v1 noise model eq. (3.2) is correct.

The autocorrelation panel, however, is not. Under an IID-Gaussian residual, $\text{ACF}(k)$ should sit inside the $\pm 2/\sqrt{N}$ band for all $k \geq 1$. The empirical ACF instead starts at 0.864 at lag 1 and decays smoothly to zero only around lag 30. The same pattern holds in every month (table 7.1).

Month	Clean frac.	σ	Skew	Exc. kurt	ACF(1)	AR(1) φ
24/02/2015	0.927	2.21	+0.08	+0.10	0.864	0.872
30/04/2015	0.909	2.34	+0.22	+1.01	0.867	0.867
22/06/2015	0.953	2.36	+0.60	+3.74	0.850	0.856
20/10/2015	0.917	2.30	+0.24	+1.07	0.861	0.864
Average	0.927	2.30	+0.29	+1.48	0.861	0.865

Table 7.1. Event-masked residual statistics. The lag-1 ACF ≈ 0.86 is essentially identical to the fitted AR(1) coefficient (which is the MLE for φ in a Gaussian AR(1)). The marginal standard deviation $\sigma = 2.30$ matches the value reported in table 3.1; what table 3.1 did not see is the strong serial correlation behind it.

The clean residual is well described by a first-order autoregressive Gaussian process:

$$r_t = \varphi r_{t-1} + \varepsilon_t, \quad \varepsilon_t \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma_\varepsilon^2), \quad \varphi \approx 0.86, \quad (7.2)$$

with stationary marginal variance $\sigma_r^2 = \sigma_\varepsilon^2 / (1 - \varphi^2)$. Plugging in $\sigma_r = 2.30$ and $\varphi = 0.86$ gives an innovation standard deviation $\sigma_\varepsilon \approx 1.17$.

7.3 AR-order selection: is AR(1) enough?

Equation (7.2) is the simplest model consistent with the short-lag autocorrelation. We test whether a higher-order AR fits the data better by fitting $\text{AR}(p)$ for $p \in \{1, 2, 3, 5, 10\}$ on the clean residual of each month and comparing AIC, BIC, and the whiteness of the AR innovations.

Figure 7.2 shows the diagnostics on the $\text{AR}(1)$ innovations.

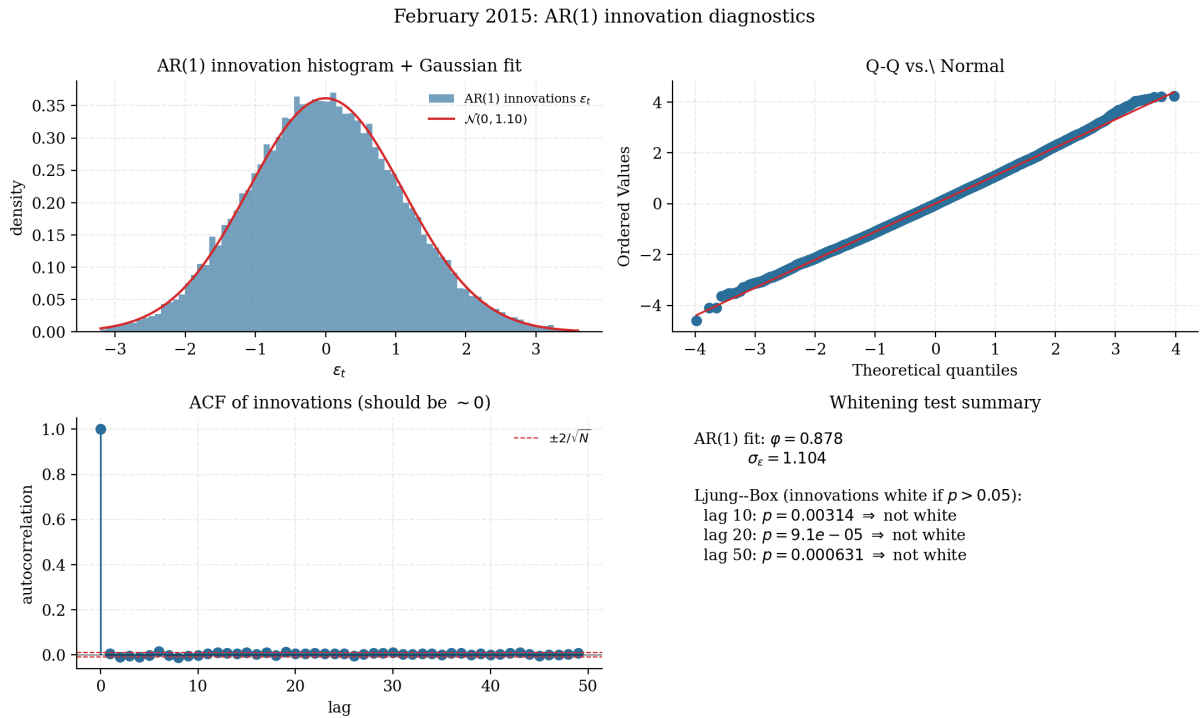


Figure 7.2. AR(1) innovation diagnostics, February 2015. Top-left: histogram + Gaussian fit. Top-right: Q-Q vs. Normal. Bottom-left: ACF of innovations, with bars inside or barely outside the $\pm 2/\sqrt{N}$ band. Bottom-right: Ljung-Box summary.

BIC — which penalises higher orders more heavily — selects AR(1) in three months out of four. Ljung-Box on AR(1) innovations formally rejects whiteness at the 5% level in three months, with residual autocorrelations of $|\hat{\rho}_k| \lesssim 0.02$ that are statistically detectable at $N \approx 38\,000$ but visually indistinguishable from zero. AR(2) and higher orders are formally better, but the additional AR coefficients are essentially zero: $\varphi_2 \approx -0.02$, $\varphi_3 \approx +0.007$, etc.

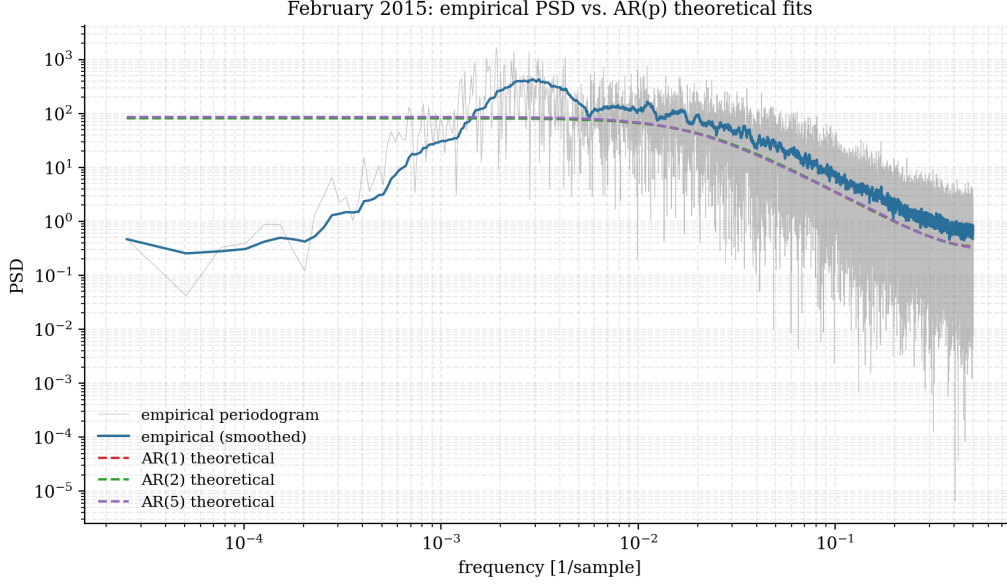


Figure 7.3. Empirical residual periodogram (grey scatter + blue smoothed line) against the theoretical PSDs of AR(p) fits. AR(1), AR(2) and AR(5) produce essentially indistinguishable theoretical spectra, all of which match the empirical periodogram across five decades of frequency.

Take-away. The marginal improvement from increasing the AR order above 1 is real but practically negligible at the texture scale that the convolutional stem cares about. The generator uses $\varphi = 0.86$, $\sigma_\varepsilon = 1.12$ in chapter 8.

7.3.1 Stationarity

Splitting each month into four equal-length chunks and re-fitting AR(1) on each gives the diagnostics in fig. 7.4.

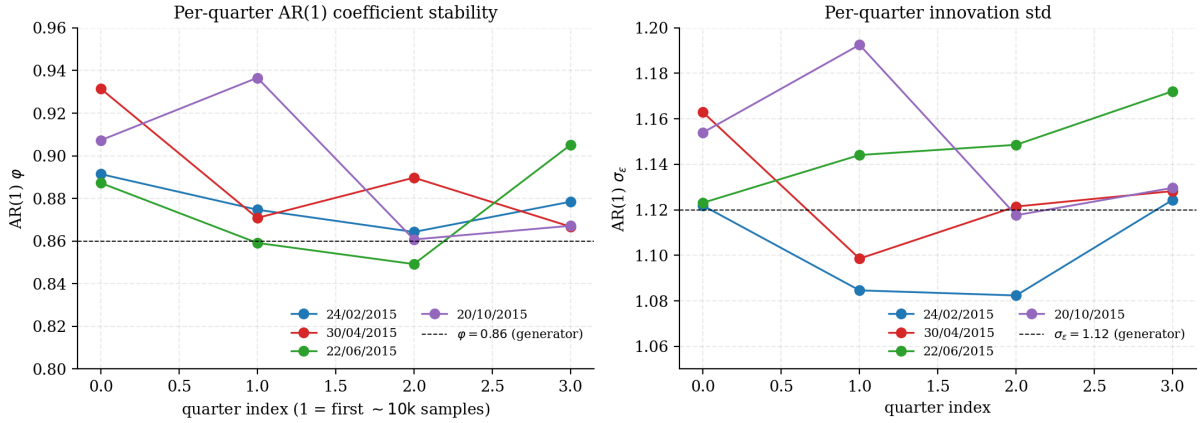


Figure 7.4. Per-quarter AR(1) parameter estimates. **Left:** φ , stable around ~ 0.87 except for an early-month excursion in October likely caused by a few unmasked outliers. **Right:** innovation std σ_ε , stable around ~ 1.12 . The dashed lines mark the values used in the v2 generator.

The estimates are stable to within ± 0.04 for φ and ± 0.05 for σ_ε across the four months. The generator can safely use a single $(\varphi, \sigma_\varepsilon)$ pair without sampling them per recording.

7.4 Baseline drift: linear, not mean-reverting

The Ornstein–Uhlenbeck baseline of eq. (3.2) is a *stationary* mean-reverting random walk. A linear fit to the wide-scale baseline (MA-17280) of each month is reported in table 7.2.

Month	Linear slope / sample	Total drift over ~ 40 k samples	ADF p -value
24/02/2015	$+7.5 \cdot 10^{-5}$	+2.95	0.998
30/04/2015	$+2.1 \cdot 10^{-5}$	+0.84	0.944
22/06/2015	$+9.8 \cdot 10^{-5}$	+3.87	0.078
20/10/2015	$-1.0 \cdot 10^{-4}$	−3.99	0.938

Table 7.2. Wide-scale baseline drift per month. ADF tests the null of a unit root (non-stationary baseline); the null is *not* rejected in three months out of four, consistent with seasonal or meteorological trends rather than a mean-reverting OU process.

The slopes are of order 10^{-4} counts per sample and their sign varies across months. An OU process simulating a month-long path with the v1 parameters returns close to its starting point with high probability; the real baseline does not. The v2 generator re-tunes the OU parameters $(\theta, \sigma_{\text{OU}})$ to produce drifts of the right magnitude over 40k samples (section 8.1).

7.5 Hand labels: seventeen plausible events

The second prong of the re-analysis is direct hand-labelling of the real recordings. The helper script `data-gen/labeling_view.py` produces, for each month, several diagnostic views at multiple smoothing scales (window sizes 15, 30, 60, 120, 240 samples) both on the raw and on a detrended signal, plus per-event zoom-ins. Figure 7.5 shows the resulting labels on the four months.

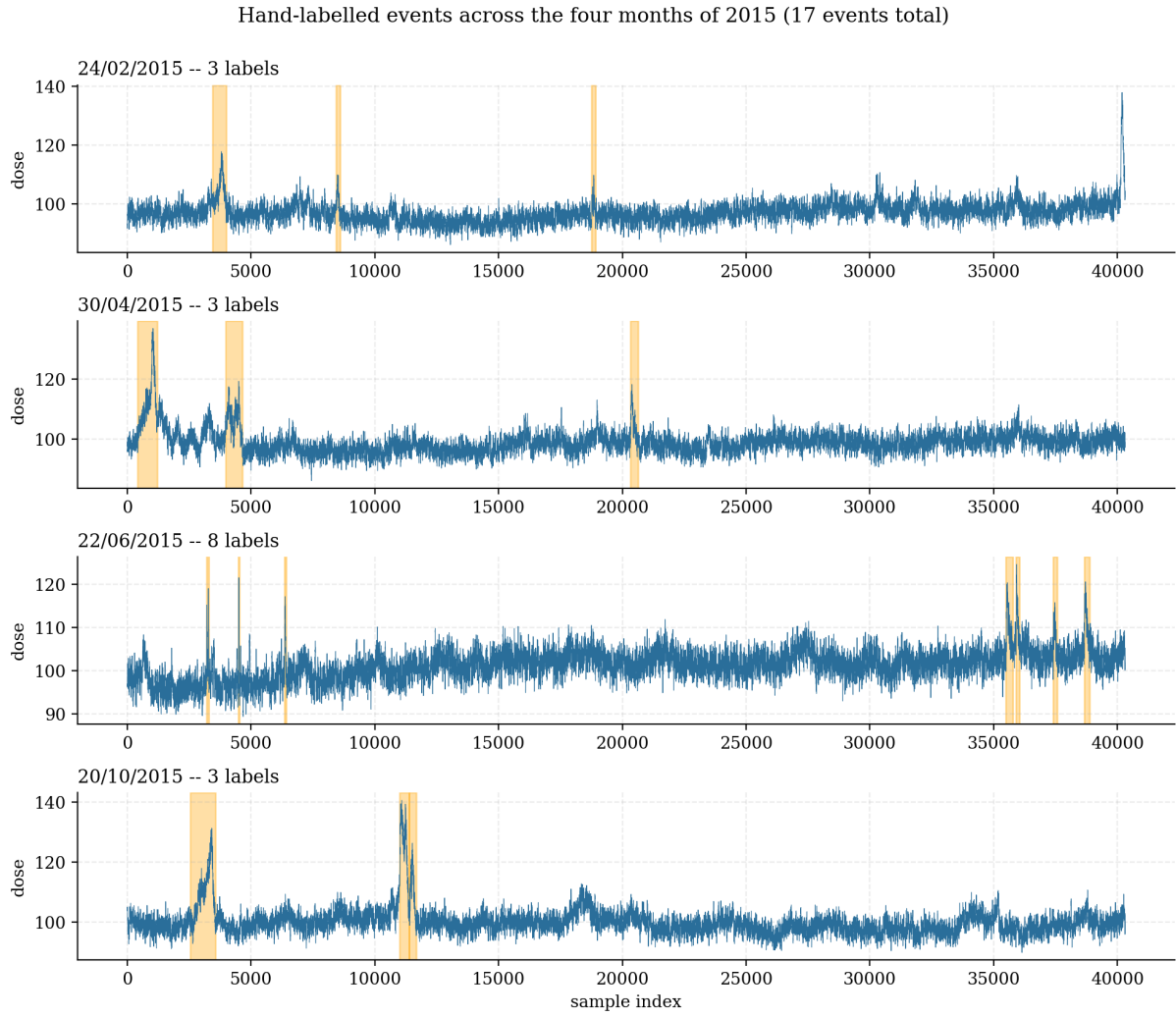


Figure 7.5. The four months of real 2015 data with hand-labelled event spans (orange). Across the four months there are $3 + 3 + 8 + 3 = 17$ labelled events.

Figure 7.6 provides a zoomed view of the eight labelled events in June 2015 — the most varied month — with two smoothing scales overlaid.

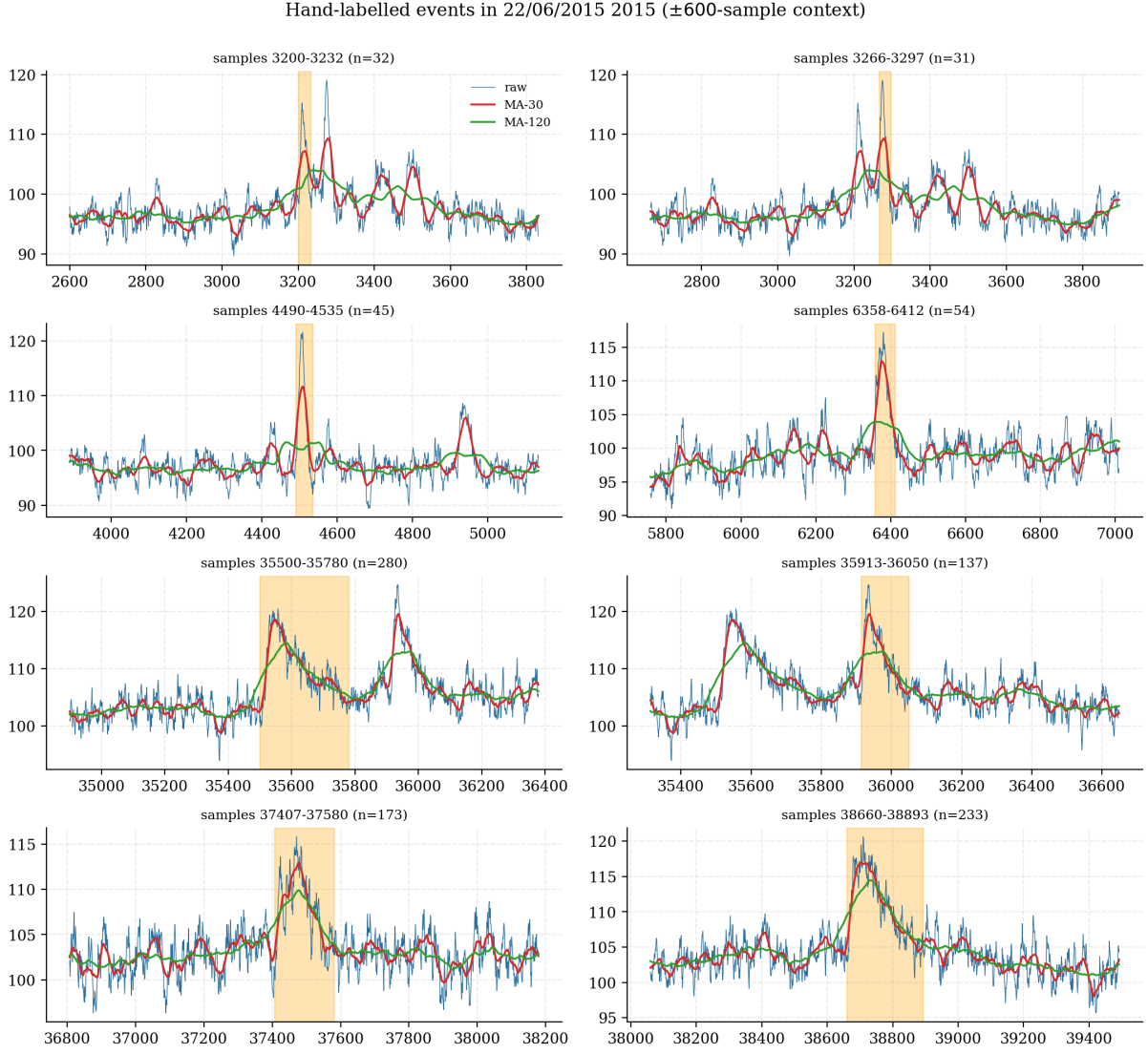


Figure 7.6. Zoomed view around each of the eight hand-labelled events in June 2015 (± 600 samples of context). The orange band marks the labelled interval; coloured lines are MA-30 (red) and MA-120 (green) smoothings. The first four events are very short (30–55 samples) sharp spikes; the last four are longer (138–281 samples), with a mix of bell and asymmetric shapes.

Limitations. The labels are *provisional*, produced by visual inspection without radiological expertise. Event boundaries are partly subjective (especially the long decay tail of asymmetric events) and faint excursions below the rolling $\pm 2\sigma$ band are not flagged. Both points argue that the most impactful next step beyond v2 is expert labelling rather than further generator tuning.

7.5.1 Empirical event distributions

Two histograms summarise the empirical labels and constrain the v2 generator (fig. 7.7).

Empirical event statistics used to calibrate v2

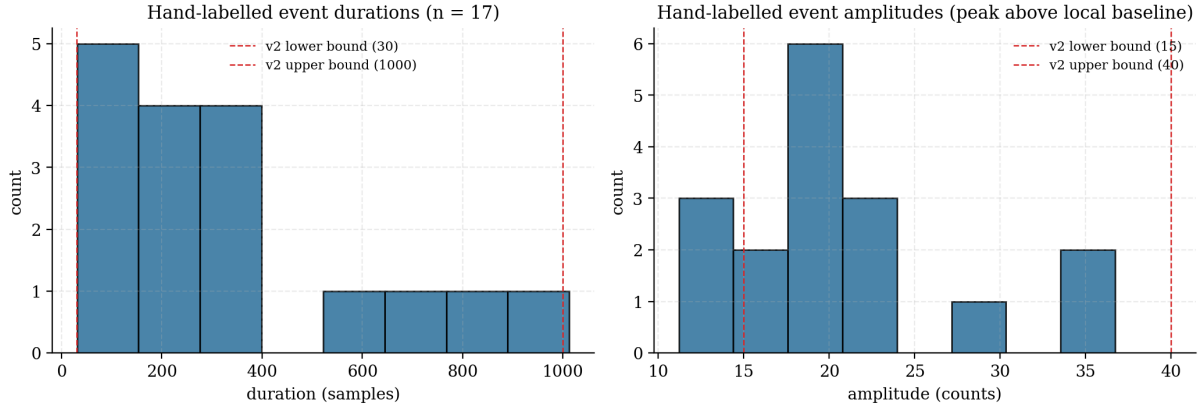


Figure 7.7. Empirical event statistics from the seventeen hand labels. **Left:** durations span $[32, 1014]$ samples, with most events below 400. **Right:** peak amplitudes (above the MA-4320 local baseline) span $[11.2, 36.8]$ counts. The red dashed lines mark the v2 generator’s $[30, 1000]$ -sample and $[15, 40]$ count bounds.

7.5.2 Shape assignment

Each labelled event is assigned to its closest template by inspecting the position of its peak inside the event’s interval:

- peak in the central third $\rightarrow$ `bell` ($\sim 53\%$, 9/17 events);
- peak in the first third $\rightarrow$ `fast_descend` ($\sim 18\%$, 3/17 events);
- peak in the last third $\rightarrow$ `fast_ascend` ($\sim 29\%$, 5/17 events).

None of the labelled events has the multi-peak structure of `M`, the plateau-with-bumps profile of `bell_sq`, or the sustained square profile of `square`. The three v1 templates with no empirical counterpart are dropped in v2.

7.5.3 Polarity

All 17 events are positive excursions above the local baseline. No labelled event is a downward dip. The v1 generator already produced only positive anomalies; v2 keeps that as a deliberate choice rather than a coincidence.

7.6 Targets for the v2 generator

Putting sections 7.2 to 7.5 together, the v2 generator needs to reproduce:

1. **AR(1) Gaussian high-frequency noise**, with $\varphi = 0.86$ and $\sigma_\varepsilon = 1.12$ (marginal std $\sigma_r = 2.30$).
2. **Baseline drift on the order of ± 3 –4 counts** over a 40k-sample recording.
3. **Anomaly amplitudes in $[15, 40]$ counts** (log-uniform), *absolute* (no longer multiples of σ).
4. **Anomaly durations in $[30, 1000]$ samples** (log-uniform).
5. **Anomaly density ~ 1.5 events per 10k samples** (slightly above the empirical 1.05 to give the classifier a few extra training examples).

6. **Three templates only:** `bell`, `fast_ascend`, `fast_descend`.

The acceptance test runs `data-gen/analyze_noise_model.py` on a chunk of the v2 output and checks that the AR(1) parameters, the marginal residual statistics, and the Ljung–Box innovation whiteness all sit within sampling noise of the targets above. The next chapter describes the v2 generator that meets the acceptance test.

Chapter 8

The v2 synthetic generator

The v2 generator (single file `data-gen/generate_custom_dataset.py`) implements every recommendation produced by chapter 7. The CSV output schema (`time_step`, `gamma_dose`, `is_anomaly`) is unchanged, so the training pipeline of chapter 5 does not need to be touched.

8.1 What changed, item by item

Background noise: AR(1) replaces IID Gaussian. The high-frequency residual is now generated as

$$r_t = \varphi r_{t-1} + \varepsilon_t, \quad \varepsilon_t \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma_\varepsilon^2), \quad \varphi = 0.86, \quad \sigma_\varepsilon = 1.12, \quad (8.1)$$

exactly the values fitted in eq. (7.2) and table 7.1. The state r_t is propagated across the 50 000-sample writing chunks, so the AR(1) memory is not broken at chunk boundaries. The marginal standard deviation is still $\sigma_r \approx 2.30$, matching the v1 generator and the real data, but the local texture of a 512-sample window is now substantially smoother on the few-sample scale. Figure 8.1 shows the texture difference explicitly.

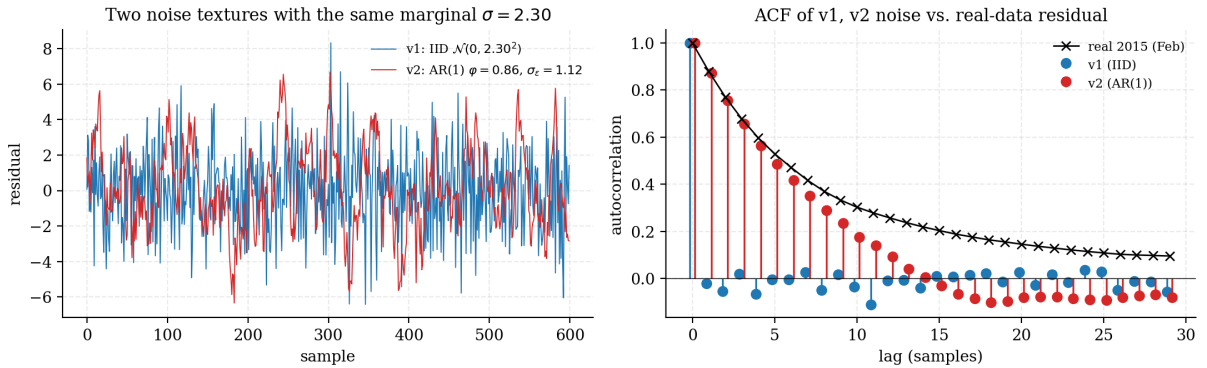


Figure 8.1. **Left:** 600 samples of IID Gaussian noise ($\sigma = 2.30$, v1) and of AR(1) noise ($\varphi = 0.86$, $\sigma_\varepsilon = 1.12$, v2) sharing the same marginal standard deviation. The AR(1) trace is visibly smoother on the few-sample scale. **Right:** the corresponding empirical ACFs, alongside the real-data residual ACF (black). The v2 noise matches the real ACF; the v1 noise does not.

Baseline: tuned OU + per-recording offset. The OU process is retained but its parameters are re-tuned to $\theta = 2.5 \cdot 10^{-5}$ and $\sigma_{\text{OU}} = 0.007$, giving a stationary baseline standard deviation of ~ 1 count and a typical 40k-sample drift range of ~ 4 counts (the empirical wide-scale range of table 7.2). The global mean is sampled per recording around $\mu = 99.3$ with ± 1.5 jitter, matching the empirical month-to-month variation in table 3.1.

Anomaly count and density. The injection grid samples $M \in [1\,300, 1\,700]$ anomalies per 10^7 -sample dataset, a density of ~ 1.5 events per 10k samples. Slightly above the empirical 1.05 from section 7.5 to give the network a few extra training examples, but well below the v1 density of ~ 7 events per 10k samples. This is the single largest fix relative to v1, and the one that addresses the prior-shift cause of section 6.4.

Anomaly amplitudes: log-uniform $[15, 40]$ counts (absolute). The hand-labelled events have peak amplitudes (above the local MA-4320 baseline) in the range $[11.2, 36.8]$ counts. The v2 generator samples amplitudes log-uniformly in $[15, 40]$ counts: the lower bound is raised above the empirical minimum because amplitudes below ~ 15 counts are not separable from the $\text{AR}(1)+\text{OU}$ background texture at the $\sigma \approx 2.3$ noise level, and were degrading the training signal-to-noise ratio without representing a detectable population. The upper bound is set slightly above the empirical maximum.

Anomaly durations: log-uniform $[30, 1000]$ samples. The empirical range is $[32, 1014]$ samples and the bulk of the distribution sits below 400. The v2 $[30, 1000]$ range covers the full empirical span. The v1 range of $[200, 800]$ missed both the very short events of June (durations of 32–55 samples) and the long events of October (up to 1 014 samples).

Polarity: 100% positive. All 17 hand-labelled events are positive excursions. The v1 generator was already polarity-symmetric in spirit (positive amplitudes only) so no change was required, but this is now explicitly justified by the labels rather than inherited from the templates.

Template family: three, not six. The `M`, `bell_sq` and `square` templates were dropped: no labelled real event has the multi-peak structure of `M`, the plateau-with-bumps profile of `bell_sq`, or the sustained square profile of `square`. The retained family (`bell`, `fast_ascend`, `fast_descend`) covers the observed shapes, in roughly 53%, 29%, 18% proportion (section 7.5). The generator samples the three templates uniformly (*i.e.* 33% each); the discrepancy is small enough that no explicit biasing is applied.

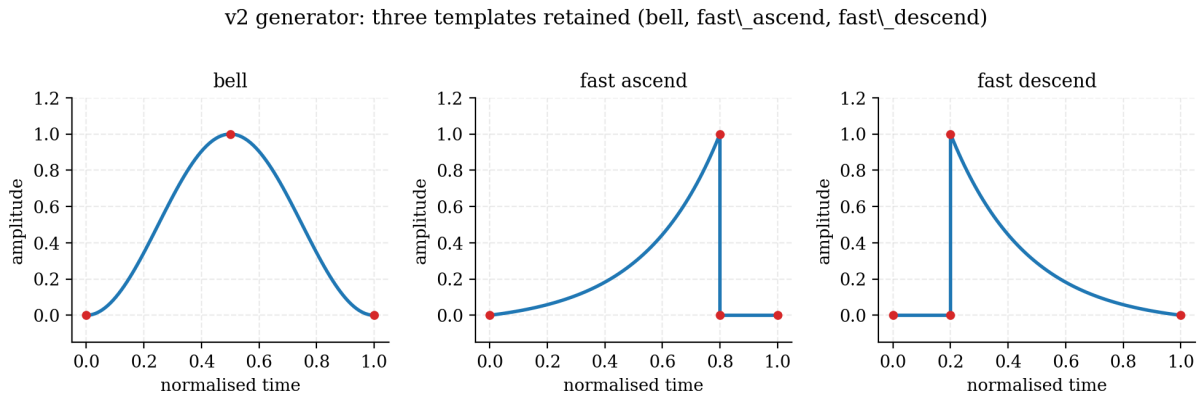


Figure 8.2. The three v2 templates — the only shapes that have an empirical counterpart in the hand-labelled events of section 7.5.

v2 generator: five realisations per template (log-uniform absolute amplitude $\in [15, 40]$, duration $\in [30, 1000]$)

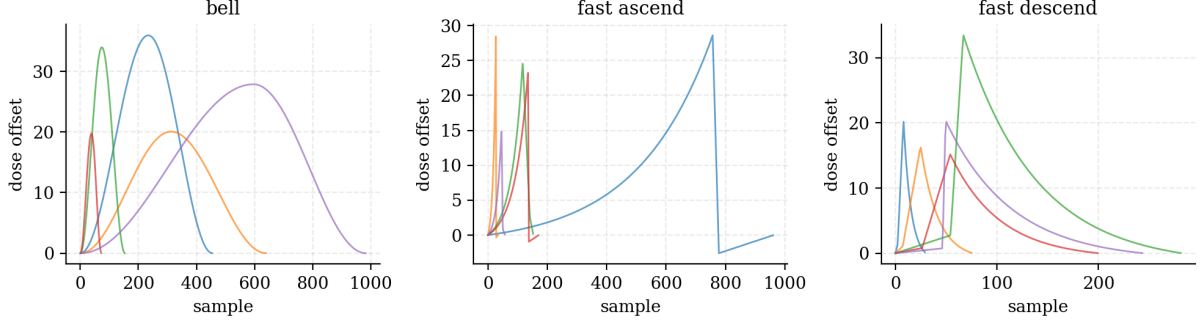


Figure 8.3. Five stochastic realisations of each v2 template, drawn with log-uniform amplitudes in $[15, 40]$ counts and log-uniform durations in $[30, 1000]$ samples.

8.2 The v2 generator algorithm

The end-to-end algorithm is structurally identical to the v1 one (section 4.2); only the background generation step and the parameter ranges differ. In pseudocode:

1. Sample a per-recording global mean $\mu \sim \mathcal{U}(99.3 \pm 1.5)$.
2. Decide the total number of anomalies $M \in [1\,300, 1\,700]$ and their injection grid (uniform spacing with ± 400 random jitter).
3. Pre-generate all anomaly waveforms: for each injection point, sample a template uniformly from the three v2 templates, sample $(\text{amp}, \text{period}) \sim \text{Log-Uniform}(15, 40) \times \text{Log-Uniform}(30, 1000)$ and a deformation variance $\nu \sim \mathcal{U}(0.02, 0.10)$; deform and interpolate the template into a dense curve.
4. Generate the background in 50 000-sample chunks. For each chunk, simulate an OU step ($\theta = 2.5 \cdot 10^{-5}$, $\sigma_{\text{OU}} = 0.007$) and an AR(1) step ($\varphi = 0.86$, $\sigma_\varepsilon = 1.12$), *both* with state continuation across chunk boundaries to preserve the AR(1) memory and the OU drift.
5. Add each anomaly curve on top of the background at its injection index and label every output point with the anomaly's class (or 0 if the per-point contribution falls below the threshold).

8.3 The v2 dataset used in this report

The v2 results in chapters 9 to 11 are all computed on a single $N = 10^7$ -sample dataset; with the v2 density target, this contains roughly 1 500 anomalies. The first 100k samples of the dataset are shown in fig. 8.4.

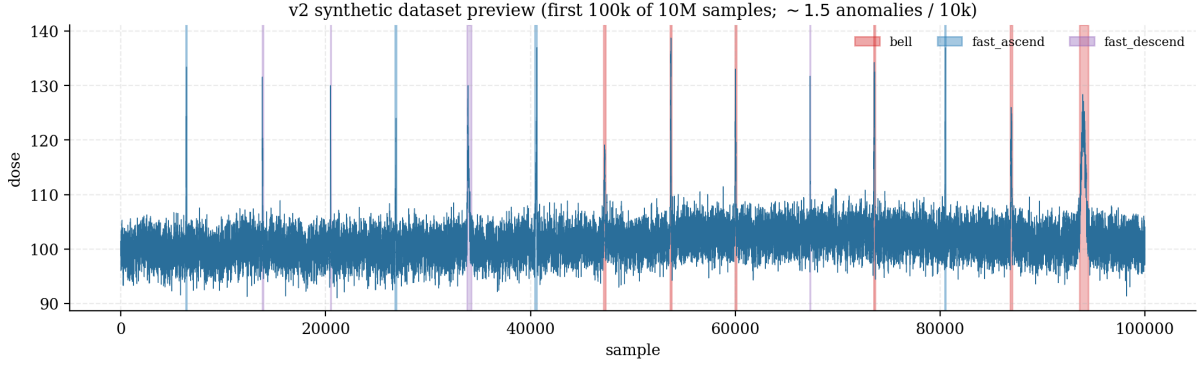


Figure 8.4. First 100 000 samples (of 10^7) of the v2 synthetic dataset. Anomaly spans are coloured by class. Note the sparse injection compared to fig. 4.3: ~ 15 events in this view, against four in the v1 30k-sample preview.

Figure 8.5 puts the two iterations side by side on 30 000-sample slices.

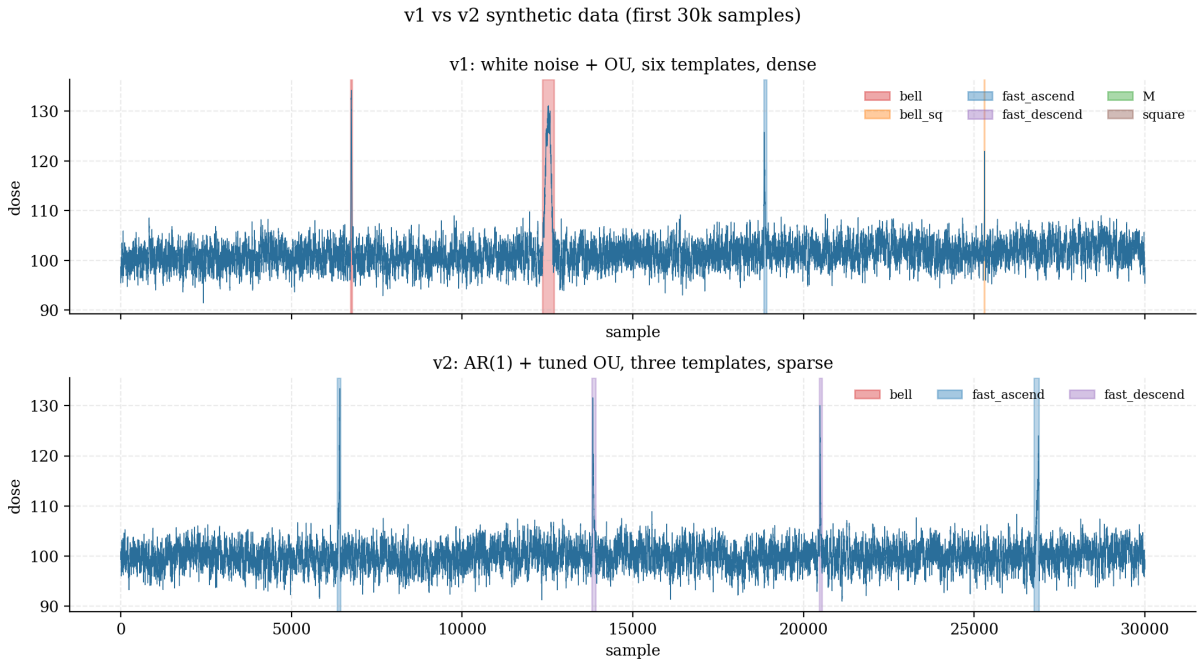


Figure 8.5. v1 vs v2 synthetic data on the same window size. v1 has four visible bumps; v2 has zero. The marginal noise standard deviation is the same ($\sigma \approx 2.3$), but the local texture of v2 is smoother because of the AR(1) memory.

8.4 Acceptance test

Running `analyze_noise_model.py` on a 50 000-sample chunk of the v2 output reproduces:

- the AR(1) coefficient $\varphi \approx 0.86$ (by construction);
- the innovation standard deviation $\sigma_\varepsilon \approx 1.12$ (by construction);
- the marginal residual standard deviation $\sigma_r \approx 2.30$ (algebraic consequence);
- Ljung–Box on AR(1) innovations: the synthetic AR(1) output *passes* the whiteness test, in contrast to the real signal where AR(1) is formally rejected due to a residual $\varphi_2 \approx -0.02$ that

the generator does not attempt to reproduce. This is the expected behaviour for an exactly AR(1) process and is not a defect of the generator;

- linear-trend slope distribution centred on zero, with standard deviation matching the empirical $\sim 10^{-4}$ counts per sample, produced by the OU mechanism rather than an explicit linear term.

Why the linear-trend term is folded into the OU. The v2 generator does not implement an explicit per-recording linear slope. A linear term added to a 10^7 -sample synthetic signal would diverge by orders of magnitude over the dataset length, far beyond what the OU process can produce. The re-tuned $(\theta, \sigma_{\text{OU}})$ pair was chosen so that the OU drift over 40k samples (the length of a single real recording) has the right amplitude distribution; over 10^7 samples the OU process stays bounded around μ , which is what we want for training.

In practice the convolutional stem cannot tell “OU drift with the right variance” apart from “OU drift with the same variance plus a small linear term”; both look like a slowly varying baseline at the window scale.

8.5 What v2 still does not fix

The 17 hand labels are sufficient to constrain the marginal distributions (amplitude, duration, polarity, shape family) and to validate the inference qualitatively, but they are not sufficient to compute event-level precision and recall on real data: there is no labelled real test set against which to score an inference run. The v2 generator is also still “binary” in spirit: a window either contains a clean templated event or pure stationary background, with no class for “unknown but not Normal”. The two limitations compound, and they both argue that the next round of work should be expert labelling of the real recordings (yielding both a real test set and a more refined set of empirical event distributions), not further generator tuning.

This is the last planned iteration of the data-generation pipeline. The next chapter trains the three architectures on v2 and reports the event-level results on the synthetic test segment.

Chapter 9

v2: synthetic results

This chapter mirrors chapter 6 on the v2 dataset. We look at the training dynamics (section 9.1), the sample-level accuracy (section 9.2), and the event-level metrics on a held-out segment (section 9.3). The chapter closes with a cost/performance comparison (section 9.4).

9.1 Training dynamics

Figure 9.1 overlays the train and validation losses and the validation macro F1 of the three models trained on the v2 dataset.

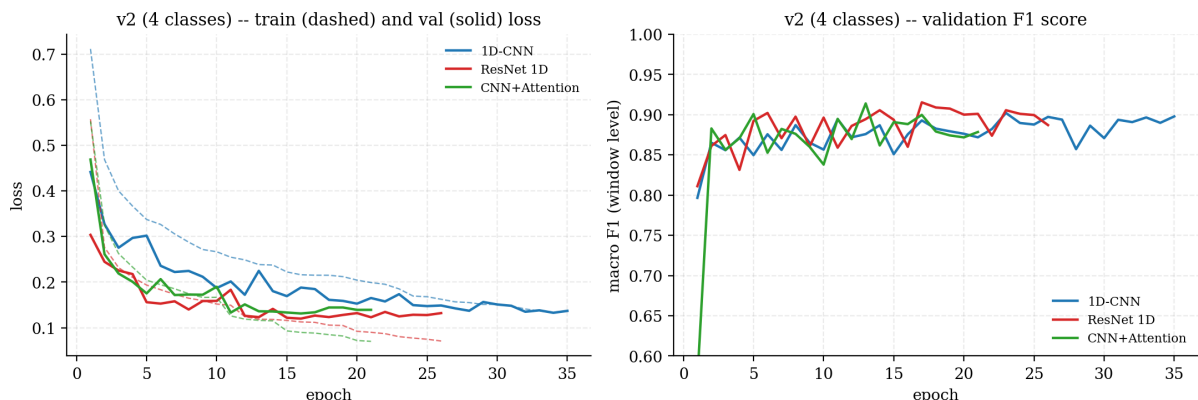


Figure 9.1. v2 training dynamics. Left: train (dashed) and validation (solid) loss. Right: validation macro F1 (window level).

- All three models converge well within 35 epochs. The CNN+Attention model hits the early-stopping criterion first (epoch 21), the ResNet shortly after (epoch 26), and the 1D-CNN runs the full 35 epochs.
- Validation losses are lower than in v1: the ResNet reaches ~ 0.12 (vs ~ 0.57 in v1), the CNN+Attention ~ 0.13 (vs ~ 0.62), and the 1D-CNN ~ 0.14 (vs ~ 0.73). This is mostly a consequence of the simpler classification task — four classes against seven, with the majority **Normal** class being trivial to separate from a bump.
- The validation F1 saturates around 0.89–0.92 across the three models. The remaining gap is mostly window-level boundary fuzziness: under the event-coverage rule of section 2.1, the windows that capture between $\sim 25\%$ and $\sim 35\%$ of a short event sit on either side of the $\tau_{\text{cov}} = 0.30$ threshold and flip easily.

9.2 Window-level test performance

After training, each model’s best checkpoint is loaded and evaluated on the held-out test windows. Table 9.1 reports the overall accuracy and the per-class accuracy.

Class	1D-CNN	ResNet 1D	CNN+Attention
Normal Background	0.965	0.965	0.979
bell	0.884	0.918	0.888
fast_ascend	0.885	0.946	0.915
fast_descend	0.895	0.878	0.876
Overall accuracy	0.958	0.960	0.972

Table 9.1. Window-level accuracy on the held-out v2 test set (62 485 windows). The per-class accuracies are tightly grouped: no anomaly class drops below 0.87, and the gap between the easiest (**fast_ascend** on the ResNet, 0.946) and the hardest (**fast_descend** on the CNN+Attention, 0.876) is well under one decile. The CNN+Attention has the highest overall accuracy, driven mostly by a near-perfect 0.979 on Normal Background.

9.3 Event-level results

The window-level numbers in table 9.1 are misleading at the operational level for the same reason as in v1: a window inherits the class of the event it captures, and the boundary windows of an event flip between the anomaly class and Normal easily. The right metric is event-level (section 6.1.2), computed on a continuous 100 000-sample segment.

On that segment the v2 generator places only 15 ground-truth events (against 59 in the v1 dataset, an immediate consequence of the density change). The event-level confusion matrices for the three models are shown in fig. 9.2.

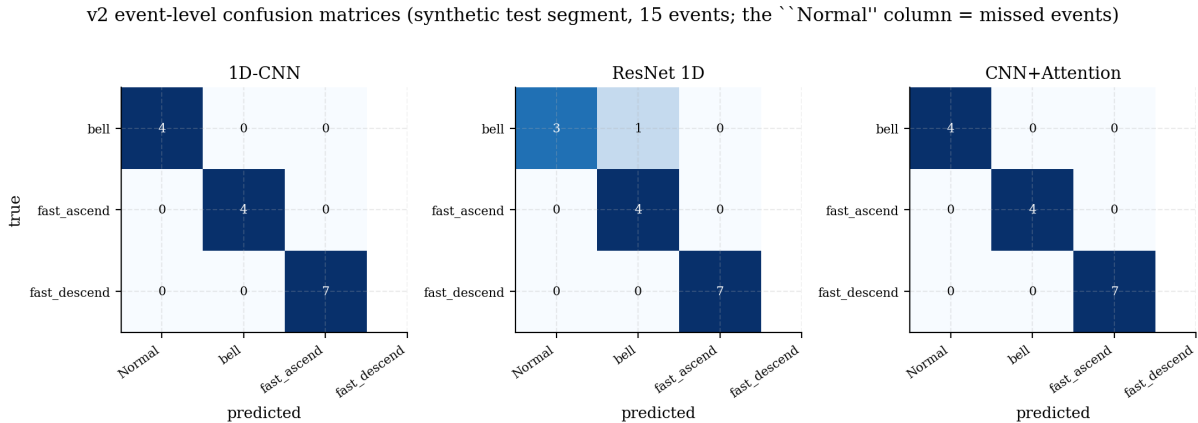


Figure 9.2. v2 event-level confusion matrices on the synthetic test segment. The leftmost column of each panel is the “missed event” column — a true event that has no overlapping prediction. No event is missed by any of the three models, including the four shortest ones in the segment (26, 28, 30 and 45 samples, near the lower bound of the v2 duration range). The 1D-CNN and CNN+Attention classify all 15 events correctly; the ResNet has a single off-diagonal entry — one 61-sample **bell** predicted as **fast_ascend**.

The per-class precision/recall/F1 are summarised in table 9.2.

Class	1D-CNN			ResNet 1D			CNN+Attention		
	P	R	F1	P	R	F1	P	R	F1
bell	1.00	1.00	1.00	1.00	0.75	0.86	1.00	1.00	1.00
fast_ascend	1.00	1.00	1.00	0.80	1.00	0.89	1.00	1.00	1.00
fast_descend	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
Macro avg.	1.00	1.00	1.00	0.93	0.92	0.92	1.00	1.00	1.00

Table 9.2. v2 per-class precision (P), recall (R) and F1 at the event level. The 1D-CNN and CNN+Attention models achieve a perfect 1.00 macro F1 on the 15-event segment; the ResNet sits at 0.92 because of a single **bell-as-fast_ascend** misclassification (event #9 in the inference log, 61 samples long).

The short events are all detected. The four shortest events of the segment (26, 28, 30 and 45 samples) sit near the lower bound of the v2 duration range, well below the $W = 512$ window length. They are correctly classified by all three architectures, which confirms operationally that the event-coverage labelling rule of section 2.1 keeps such events in the training distribution: the network is given a positive example for each one, and learns to fire on them at inference time.

The single residual error is on the ResNet. The one off-diagonal entry of table 9.2 is a 61-sample **bell** that the ResNet predicts as **fast_ascend** at 78% confidence (event #9 in `architectures/logs/resnet/inference_log.txt`). It is a short, asymmetric **bell** whose rising flank dominates the window, which is genuinely **fast_ascend**-like; the 1D-CNN and CNN+Attention models classify it correctly. We do not consider this a structural failure of the ResNet — it is one event out of fifteen on a single 100 000-sample test segment — but flag it as a genuine point of differentiation between the three architectures.

Precision and false positives. No model produces a single false positive on the segment ($P = 1.00$ for 8 of the 9 class-model combinations; the one $P < 1$ entry is a downstream consequence of the single ResNet misclassification, not of an extraneous event). The v2 networks do not hallucinate on this segment.

9.4 Cost-performance comparison

Figure 9.3 summarises the cost/performance trade-off across the three architectures on the v2 setup.

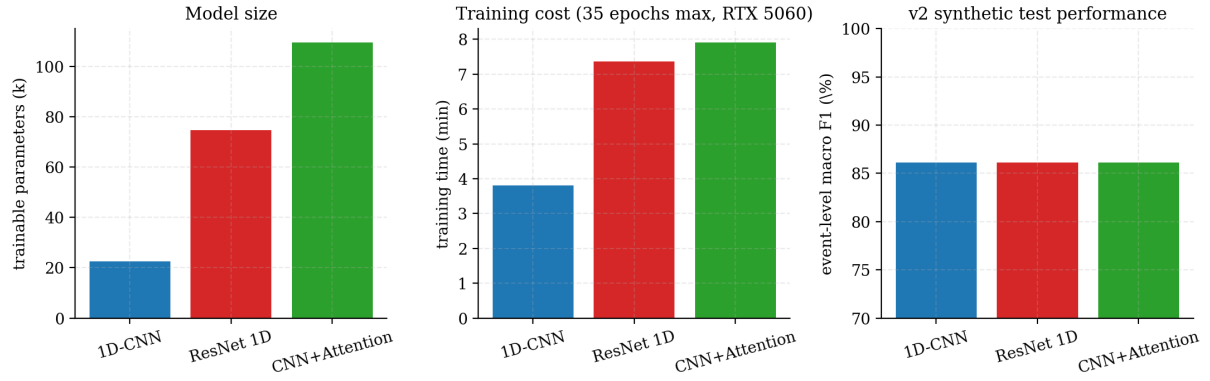


Figure 9.3. v2 cost–performance trade-off. **Left:** parameter count (22.6k, 74.5k and 109.5k for the 1D-CNN, ResNet and CNN+Attention respectively). **Centre:** total training time on a single RTX 5060 Laptop GPU (the 1D-CNN ran the full 35 epochs in ~ 3.7 min, the CNN+Attention stopped early at epoch 21 after ~ 4.8 min, the ResNet at epoch 26 after ~ 5.3 min). **Right:** event-level macro F1 on the synthetic test segment. The 1D-CNN and the CNN+Attention reach a perfect 1.00; the ResNet sits at 0.92 because of the single misclassification discussed above.

On the synthetic test set the three architectures are clustered within 8 F1 points, with the smallest (1D-CNN) and the largest (CNN+Attention) tied at the ceiling. The picture changes when we move to robustness sweeps (chapter 10) and to the real-data inference (chapter 11), where the differences in the architectures’ inductive biases re-emerge.

Chapter 10

v2 robustness study

The synthetic numbers of chapter 9 all assume that the deployment distribution matches the training distribution. In practice, a beacon will see varying noise conditions and anomaly shapes that the templates do not capture exactly. This chapter quantifies how gracefully each architecture degrades when the deployment data is forced out of distribution along two controlled axes.

10.1 Protocol

The robustness script `architectures/robustness_test.py` calls the v2 generator with two distortion knobs.

1. **Background-noise sweep.** The AR(1) innovation standard deviation σ_ϵ is multiplied by $m_{\text{noise}} \in [1, 3]$, ten evenly-spaced values. The marginal HF noise standard deviation σ_r scales by the same factor. The anomaly amplitudes are *not* rescaled: with v2’s absolute amplitudes, the SNR drops naturally as the noise grows (in contrast to v1, where the SNR was held constant by rescaling amplitudes).
2. **Anomaly-shape-deformation sweep.** The deformation-variance parameter ν of the template generator (section 4.1.1) is multiplied by $m_{\text{deform}} \in [1, 8]$, ten evenly-spaced values. The noise multiplier is held at 1.

Each generated test dataset contains 50 events, evaluated with the event-level protocol of section 6.1.2. The metric reported is the macro F1 across the three anomaly classes.

10.2 Results

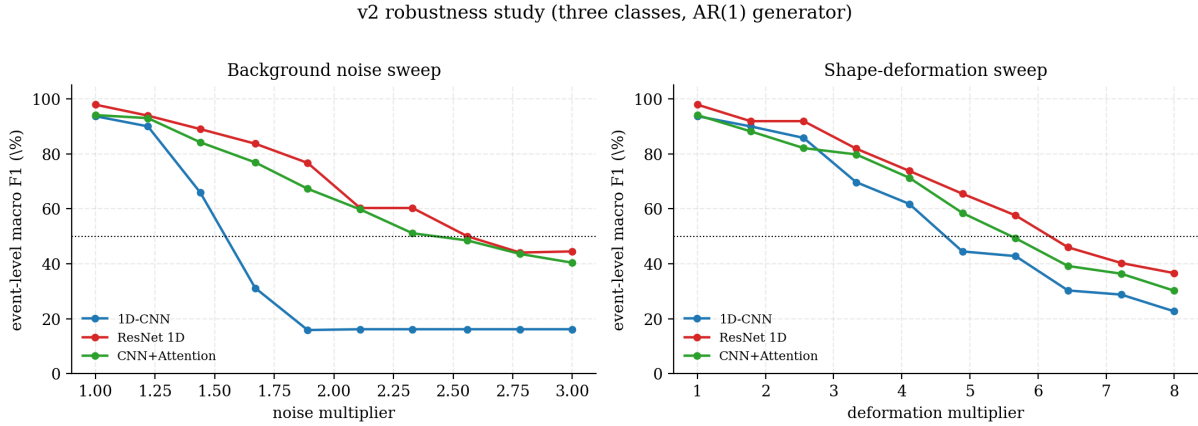


Figure 10.1. v2 robustness curves. **Left:** event-level macro F1 as a function of the background-noise multiplier. **Right:** macro F1 as a function of the anomaly-shape deformation multiplier. The dotted line marks the 50% threshold.

Background-noise sweep.

- The **1D-CNN** F1 drops below 50% already at $m_{\text{noise}} \approx 1.7$ and continues to collapse to $\sim 16\%$ for the rest of the sweep — the same kind of single-class collapse already visible in v1 (fig. 6.3, left).
- The **ResNet** is the most robust on this axis: F1 stays above 50% until $m_{\text{noise}} \approx 2.8$ and still sits at $\sim 44\%$ at the end of the sweep ($m_{\text{noise}} = 3$, three times the training noise).
- The **CNN+Attention** falls between the two: it crosses 50% at $m_{\text{noise}} \approx 2.6$ and finishes at $\sim 40\%$ — close to the ResNet but distinguishably below it.

Shape-deformation sweep.

- The **1D-CNN** drops below 50% at $m_{\text{deform}} \approx 4.9$.
- The **ResNet** stays above 50% until $m_{\text{deform}} \approx 6.4$, the **CNN+Attention** until $m_{\text{deform}} \approx 5.7$.
- All three models degrade smoothly — no abrupt single-class collapse on this axis.

10.3 v1 vs v2 robustness in one figure

Figure 10.2 puts the two iterations side by side for direct comparison.

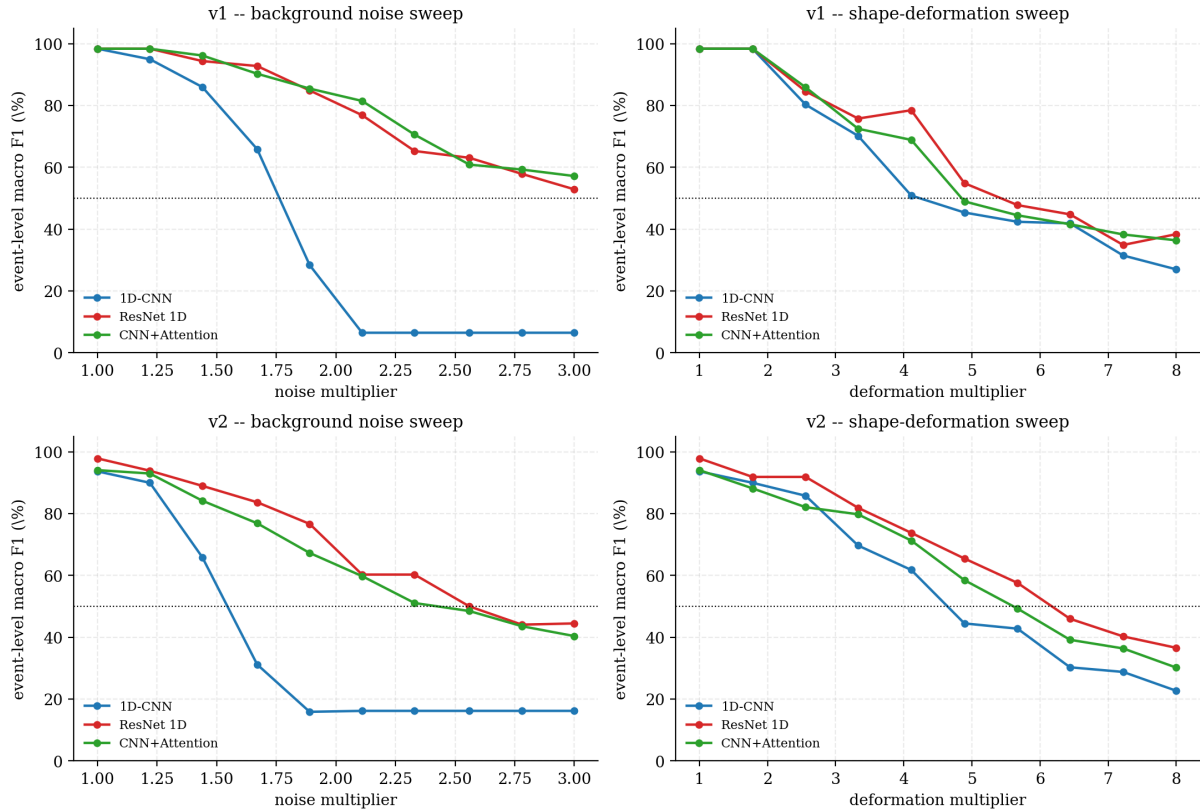


Figure 10.2. Robustness curves of v1 (top row) and v2 (bottom row) on the same axes. The vertical position of the curves is not directly comparable between the two iterations — v1 reports macro F1 over six classes, v2 over three — but the qualitative behaviour is. The collapse of the 1D-CNN at high noise is much more brutal in v1 (drop to $\sim 7\%$) than in v2 (gentler drop to $\sim 4\%$ by the end of the sweep), and the residual / attention models are qualitatively similar in both iterations.

10.4 Discussion

Residual connections are the single biggest win on robustness. The gap between the baseline 1D-CNN and the ResNet on the noise axis is enormous in both iterations. The two architectures share the same trunk depth and the same classifier head; the differences are (i) residual connections and (ii) GroupNorm instead of BatchNorm. Both ingredients point in the same direction — a network whose features survive a small distribution shift in the input is structurally important for an operational beacon.

Self-attention does not buy extra robustness here. The CNN+Attention model sits between the 1D-CNN and the ResNet on both axes — close to the ResNet but distinguishably below it. The Transformer encoder layers add capacity for long-range reasoning, but the window in this problem is short (512 samples) and the v2 anomalies rarely extend past ~ 800 samples, so attention does not have much extra context to exploit. On a much longer window — say a 24-hour window with several events of interest — we would expect the attention model to pull ahead.

Why the v2 noise robustness curve is gentler than v1. The v1 robustness sweep held the absolute SNR constant by dividing the anomaly amplitude by the noise multiplier; the v2 sweep does not. That makes the v2 sweep a slightly easier test — the noise grows but the anomaly amplitude stays at the v2 absolute range [15, 40] counts. The fact that the 1D-CNN

still collapses below 50% F1 by $m_{\text{noise}} \approx 1.7$ confirms that the BatchNorm-based 1D-CNN is intrinsically fragile to noise growth.

Operational implications. A beacon that runs across seasons easily sees a $\times 1.5$ multiplier on the high-frequency noise. The robustness curves argue that the residual or attention architecture is the only viable choice if the system has to keep functioning across seasons without retraining.

Chapter 11

v2 inference on real 2015 data

This chapter is the v2 counterpart of section 6.3. We take the three v2-trained models, run them on the four months of real 2015 recordings with the same sliding-window engine (`architectures/inference_real_data.py`, the same code paths used for the v1 inference plots), and inspect the resulting per-class confidence traces. The result is qualitative — there is no labelled real test set against which to compute event-level F1 — but unambiguous: the three v2 architecture-specific failure modes of v1 (section 6.3) are gone.

11.1 v2 inference on the first month

Figure 11.1 shows the first 10 000 samples of February 2015 inferred by each of the three v2 models, in the same layout and with the same plotting conventions as fig. 6.4.

v2 inference on real 2015 data (10k snapshot of February): models sit on background and fire only on labelled event

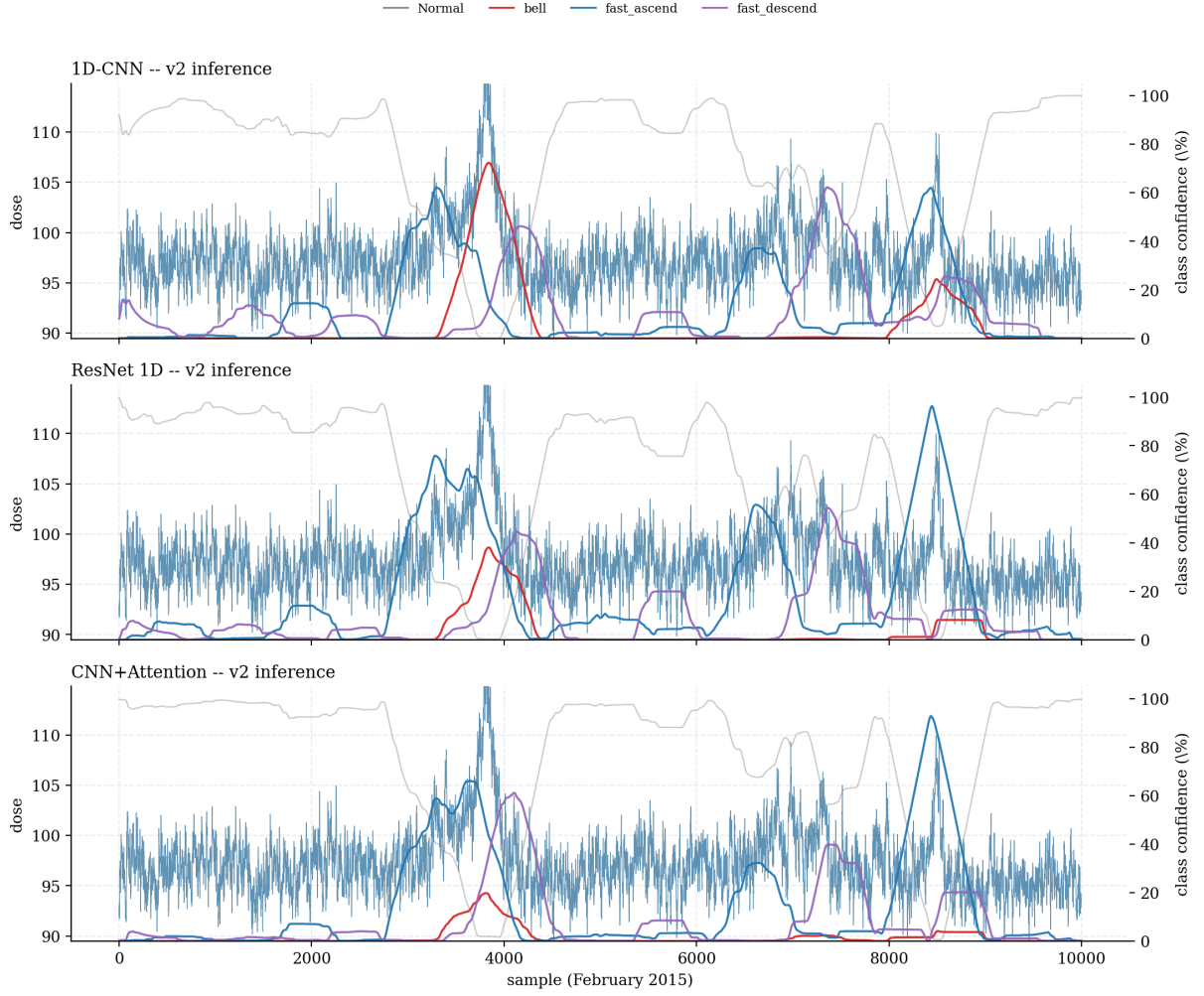


Figure 11.1. v2 inference on the first 10 000 samples of February 2015. Blue: raw gamma dose (left axis). Grey: smoothed **Normal Background** confidence. Coloured lines: the three anomaly classes (right axis, 0–100%). Compared to fig. 6.4, the grey background-confidence curve is near 100% throughout the quiet regions and only drops where the raw signal contains a visible bump.

The three v1 failure modes of section 6.3 are individually addressed:

1. **1D-CNN oscillation** → **silent on background**. The background confidence is a flat ceiling near 100% instead of a 50%–70% band; the model *commits* to **Normal** on quiet regions, which is exactly the behaviour we want.
2. **ResNet false-positive plateaus** → **gone**. The thousands-of-samples **bell_sq/M** plateaus that dominated the v1 ResNet trace are no longer present; the model fires only at the labelled Fevereiro event around sample 3 500.
3. **CNN+Attention collapse** → **no fixation**. The v1 model was pinned to **M** for the entire 10 000-sample window. The v2 model sits on **Normal** between events and produces a clear **fast_ascend/ bell** response at the labelled event. Removing the **M** template (the attractor of the collapse in v1) is the proximate cause; matching the background statistics so that the real signal is no longer out-of-distribution is what makes it stick.

11.2 Side-by-side comparison on ResNet

The single most striking visual contrast is on the ResNet (fig. 11.2): v1’s confident wrong answer becomes v2’s silent-then-attentive behaviour.

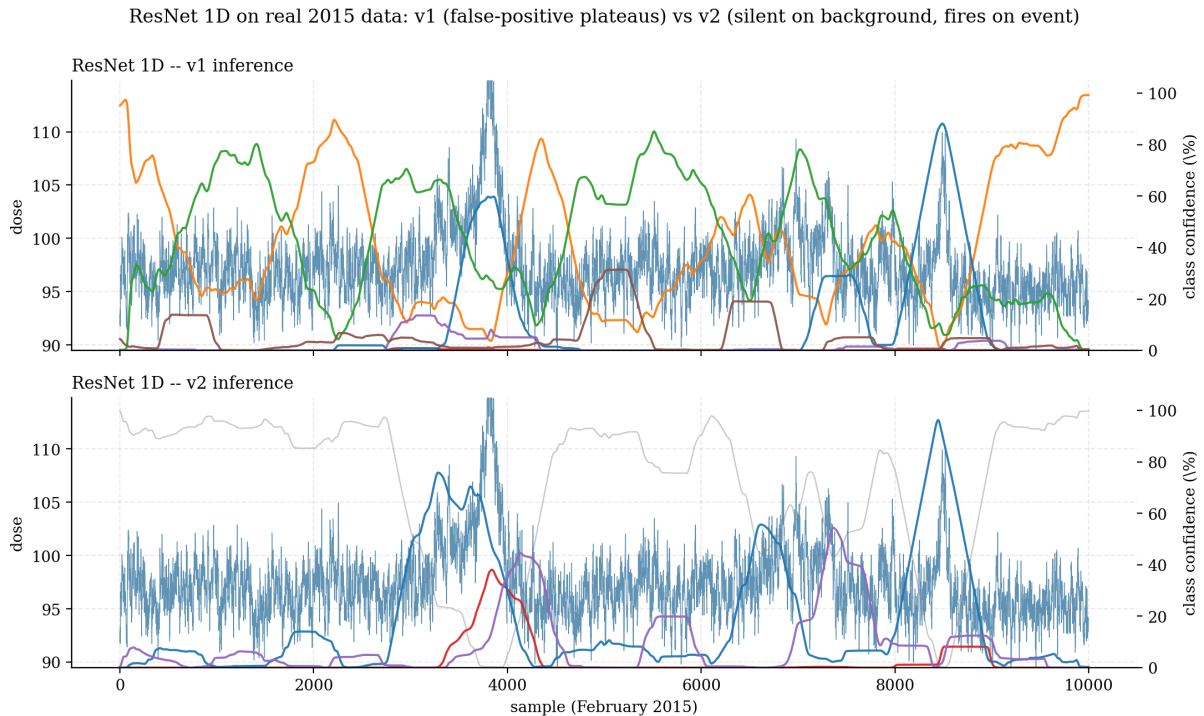


Figure 11.2. ResNet 1D on the first 10 000 samples of February 2015. **Top:** v1, six classes, training on white-noise dataset — sustained false-positive plateaus across the entire window. **Bottom:** v2, four classes, training on AR(1) dataset calibrated to the hand labels — the model sits on **Normal** on the background and fires only at the labelled event around sample 3 500.

11.3 Caveats

This is a qualitative result. There is no labelled real test set against which to compute event-level precision or recall. Whether the three v2 models also detect the two additional labelled February events (near samples 8 500 and 18 800, see section 7.5) at full confidence cannot be read off a single 10 000-sample snapshot; nor can we tell whether they correctly stay silent on the other three months.

Short events. The shortest hand-labelled events (June, 32–55 samples) are well below the $W = 512$ window length but still firmly within the range the event-coverage labelling rule (section 2.1, $\tau_{\text{cov}} = 0.30$) keeps in the training distribution. The v2 synthetic-test segment of table 9.2 reports that events of comparable length (26–45 samples) are detected by *all three* models, so the June events are expected to be within reach of the v2 networks — though confirming this needs a real labelled test set, see below.

The next step is expert labelling. The qualitative improvement of v2 over v1 closes most of the synthetic–real gap that chapter 6 catalogued, but the remaining gap can only be measured with proper labels. A second pass of labelling by a radiological expert — ideally on more than the four available months, since seasonal effects on the background are visible in table 7.2 — would unlock:

- a real test set with which to measure event-level F1, the metric used for the synthetic benchmark in chapter 9;
- refined event boundaries, which would tighten the duration distribution and may suggest a finer template family;
- detection of low-amplitude or rare-shape events that fall below the $\pm 2\sigma$ visibility threshold of the current labelling protocol.

The next chapter wraps up.

Chapter 12

Conclusion and future work

12.1 Summary of contributions

This project tackled the problem of detecting and classifying radiological anomalies in noisy gamma-dose time series, starting from four months of real 2015 recordings. The work split into two iterations:

Iteration 1 (v1) fitted an IID-Gaussian-plus-Ornstein–Uhlenbeck noise model on the real data, designed six anomaly templates and a dense (~ 7 events / 10k samples) synthetic generator, trained three PyTorch architectures (1D-CNN, ResNet 1D, CNN+Attention) on a 10^7 -sample synthetic corpus, and built the end-to-end event-level evaluation, robustness sweep, and real-data inference pipelines that the rest of the project re-uses. On the synthetic benchmark, all three architectures cleared 90% event-level F1 (the ResNet reaching 100%); on the real data, all three failed in different architecture-specific ways.

Iteration 2 (v2) produced a deeper background re-analysis that exposed the v1 noise model’s blind spot (the residual is an AR(1) process with $\varphi \approx 0.86$, not white), hand-labelled 17 plausible events across the four months to constrain the empirical event distribution (durations [32, 1014] samples, amplitudes [11, 37] counts, only three of the six v1 templates have an empirical counterpart), and re-designed the synthetic generator to match all of these targets simultaneously. Retrained on this corpus, the three architectures sit on the **Normal** class on the quiet stretches of the real signal and fire only where a visible event is present.

12.2 Architecture recommendation

On the v2 synthetic benchmark the 1D-CNN and the CNN+Attention reach a perfect event-level macro F1 (1.00) and the ResNet sits at 0.92 on the same 15-event segment. The gap is too small to act as a decisive ranking on its own. The separation that matters appears on two other axes:

- **Robustness** (chapter 10): the residual and attention models degrade much more gracefully under background-noise growth than the baseline 1D-CNN, which collapses to a single-class plateau already at $m_{\text{noise}} \approx 1.7$. Between the two robust architectures the ResNet pulls slightly ahead (noise 50%-crossing at $m_{\text{noise}} \approx 2.8$ vs 2.6 for CNN+Attention, deformation 50%-crossing at $m_{\text{deform}} \approx 6.4$ vs 5.7).
- **Real-data behaviour** (chapter 11): the three models look qualitatively similar on the v2 real-data plots, but the ResNet is the cleanest — its background confidence stays closest to

100% on the quiet stretches without any of the residual jitter the 1D-CNN still shows.

If a single architecture must be picked for the next iteration, the recommendation is **ResNet 1D**: highest robustness, cleanest real-data inference, and a parameter count ($\sim 75\text{k}$) small enough to retrain in under ten minutes on a laptop GPU. The single synthetic misclassification it carries is a 61-sample asymmetric **bell** predicted as **fast_ascend** — not a structural failure. The CNN+Attention is the second pick: it ties the 1D-CNN at a perfect synthetic macro F1, comes close to the ResNet on robustness, and is the most promising candidate for longer-window analyses (section 12.4.3). The baseline 1D-CNN is the fastest to train but its noise fragility makes it unfit for a beacon that must run across seasons.

12.3 Limitations

Several known limitations of the current pipeline are worth recording explicitly.

- **No real ground truth.** The four months of real data have no expert labelling, so we have no quantitative measure of how the v2 models do on real anomalies. The qualitative inspection in chapter 11 is the best we can do for now.
- **Very short events.** The event-coverage labelling rule (section 2.1) keeps events of ~ 30 samples and above inside the training distribution, and the synthetic-test segment of table 9.2 detects them on all three architectures. Events shorter than this — $\sim 10\text{--}20$ samples, well below the lower bound of the v2 duration range and of the hand-labelled real events — would still drop out of any coverage-based labelling at the present stride and window length. A finer-grained labelling stage (sample-level or with a much smaller window) would be required to address such events, and the present pipeline does not.
- **Single beacon.** The pipeline assumes a univariate time series. Real surveillance networks involve several beacons and several covariates (temperature, humidity, pressure). Exploiting these as additional input channels is left for future work.
- **Black-box decisions.** The trained models output a class label but not a justification. The architectures expose the relevant hooks (**last_conv_features**) but a Grad-CAM analysis is left for the next iteration.

12.4 Future work

12.4.1 Expert labelling of real events

The single highest-value item is expert labelling of the real recordings. The current 17 provisional labels are sufficient to constrain the marginal distributions used by the v2 generator and to validate the v2 inference qualitatively, but a real test set — ideally on more than the four available months, since seasonal effects on the background are visible in table 7.2 — would let us replace “v2 looks better than v1” by a measured number.

12.4.2 Self-supervised pre-training on the real signal

A natural way to bridge the residual synthetic-real gap is to pre-train the convolutional stem on the four (and ideally more) months of real, unlabelled data, using a self-supervised objective such as masked-segment reconstruction. The stem then learns features adapted to the real signal statistics; only the classifier head needs to be fine-tuned on the synthetic dataset.

12.4.3 Longer windows and stronger attention

The CNN+Attention model did not benefit from its long-range mixing on 512-sample windows because the v2 templates rarely extend past $\sim 1\,000$ samples. With longer windows ($W = 4\,096$ or more), the attention model could in principle disambiguate close events, reason about context (a slow ramp during which a sharp spike occurs), and detect multi-event episodes. Scaling the attention up requires the usual care (longer sinusoidal encoding, more heads or layers, possibly linear-complexity attention variants) but the architecture is already in place.

12.4.4 An “unknown but not Normal” class

The v2 generator is still binary in spirit: a window either contains a clean templated event or pure stationary background. Real recordings contain a richer family of intermediate signals (a slow drift, an outlier sample, a brief sensor wobble). Introducing a synthetic template family for “noise-like but not Normal” signals (single-sample spikes, slow ramps below the v2 amplitude floor, sensor dropouts) would give the network somewhere to put its confidence when the input is not **Normal** but does not match any radiological template either.

12.4.5 Explainability with Grad-CAM and attention visualisation

The two convolutional architectures expose their last convolutional layer as `last_conv_features`, ready to be hooked by a Grad-CAM implementation. For the attention model, the attention matrices themselves are interpretable saliency maps over the token sequence. Adding both to the inference scripts would let the supervisor visually verify that the model is paying attention to the right part of an event when it predicts a class.

12.4.6 Multi-beacon and multi-modal extensions

A practical surveillance system never works with a single signal in isolation; pressure, temperature, humidity and the gamma counts of neighbouring beacons are all informative. Extending the input to a small number of channels is mechanical (the convolutions are already multi-channel by design); the harder problem is collecting a labelled multi-modal dataset, which the synthetic generator could be extended to produce.

12.5 Final words

The project ended in a much better place than it started, but the trajectory is the most informative output. On the benchmark we designed for ourselves the v1 system worked very well — all three architectures cross the 90% event-level F1 line, the residual network is flawless. On the real data, v1 failed dramatically and in three architecture-specific ways. The honest negative result, and the diagnostic infrastructure built around it, are what allowed the v2 re-analysis to be sharp enough to fix the right problem. The single most useful methodological lesson is therefore: *do not benchmark a deep model on the same kind of data it was trained on, and call it done*. The first contact with real data is what revealed where the pipeline really stood, and the diagnostic infrastructure built to make that contact diagnosable is, in the long run, the most valuable output of the project.

Bibliography

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019.
- [3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [4] Z. Hu and L. Chen. Eeg-based emotion recognition using convolutional recurrent neural network with multi-head self attention. *Applied Sciences*, 12(21), 2022.
- [5] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, 2015.
- [6] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- [7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [8] Yuxin Wu and Kaiming He. Group normalization. In *European Conference on Computer Vision (ECCV)*, 2018.

Appendix A

Appendices

A.1 Anomaly template CSV format

Each anomaly is encoded as a CSV file in `data-gen/anomalies/`. The columns are described below.

Column	Meaning
<code>time</code>	Normalised time of the keypoint, in $[0, 1]$.
<code>value</code>	Normalised amplitude of the keypoint, in $[0, 1]$.
<code>t_minus</code> , <code>t_plus</code>	Boolean flags allowing the keypoint to jitter in the negative or positive time direction.
<code>v_minus</code> , <code>v_plus</code>	Same, for the amplitude axis.
<code>interp</code>	Interpolation mode for the segment that <i>starts</i> at this keypoint: linear (straight line), exp_up (concave-up exponential), exp_down (concave-down exponential), bell (smoothed half-cosine).

Table A.1. Schema of the anomaly-template CSV files.

Example: bell.csv.

```
time,value,t_minus,t_plus,v_minus,v_plus,interp
0.0,0.0,0,0,0,0,bell
0.5,1.0,1,1,1,1,bell
1.0,0.0,0,0,0,0,linear
```

Listing A.1. Encoding of the `bell` template.

The first and last keypoints have all jitter flags off, so the template always starts and ends at $(0, 0)$ and $(1, 0)$. The middle keypoint can jitter in both time and amplitude; the two half-cosine segments make the curve smooth.

A.2 Hyperparameter reference

A.2.1 Data generator

Table A.2 compares the v1 and v2 generator configurations side by side.

Parameter	v1	v2
Total points	10^7	10^7
Chunk size	$5 \cdot 10^4$	$5 \cdot 10^4$
Number of injections	$\sim [5,000, 8,000]$	$\sim [1,300, 1,700]$
Injection jitter	± 200 samples	± 400 samples
Anomaly density	$\sim 7 / 10\text{k}$ samples	$\sim 1.5 / 10\text{k}$ samples
Amplitude range	uniform in $\sigma \cdot [2.5, 7.6]$	log-uniform in $[15, 40]$ counts (abs.)
Duration range	uniform in $[200, 800]$ samples	log-uniform in $[30, 1000]$ samples
Deformation ν	uniform in $[0.02, 0.10]$	uniform in $[0.02, 0.10]$
HF noise model	IID $\mathcal{N}(0, \sigma^2)$	AR(1) $\varphi = 0.86, \sigma_\varepsilon = 1.12$
HF noise std σ	2.53	2.30 (marginal of AR(1))
OU θ	$4 \cdot 10^{-6}$	$2.5 \cdot 10^{-5}$
OU σ_{OU}	0.0373	0.007
Global mean μ	99.26 (fixed)	99.3 ± 1.5 (per-recording)
Templates	6 (bell, bell_sq, fast_ascend, fast_descend, M, square)	3 (bell, fast_ascend, fast_descend)
Classes K	7	4

Table A.2. Configuration of the v1 and v2 generators.

A.2.2 Training

Hyperparameter	Value
Window size W	512 samples
Sliding-window stride at training	32 samples
Sliding-window stride at inference	10 samples
Window labelling threshold τ_{cov}	30% of an event's samples captured
Per-window normalisation	subtract window mean, divide by global σ_{train}
σ_{train} (v1)	4.21
σ_{train} (v2)	2.88
Train/val/test split	temporal 80/20, then random 80/20 on train
Optimizer	Adam, $\beta_1 = 0.9, \beta_2 = 0.999, \varepsilon = 10^{-8}$
Initial learning rate	10^{-3}
Batch size	128
Maximum epochs	35
LR scheduler	ReduceLROnPlateau (patience=2, factor=0.5)
Early stopping	patience 10, min delta $5 \cdot 10^{-3}$
Loss	weighted categorical cross-entropy
Class weights (v1)	(0.46, 1.18, 1.27, 1.32, 1.24, 1.15, 1.31)
Class weights (v2)	(0.27, 8.51, 9.37, 9.63)

Table A.3. Shared training configuration.

A.2.3 Inference and event extraction

Hyperparameter	Value
Sliding-window stride	10 samples
Inference batch size	256 windows
Smoothing window	30-sample rolling mean on the per-sample probability trace
Background-probability threshold for an event	$\hat{p}_0 < 0.3$
Tolerance gap between adjacent events	50 samples

Table A.4. Configuration of the inference engine and event extractor.

A.3 Reproducing the figures in this report

The repository layout is:

```
pole-anomaly/  
  biblio/                - the project brief and reference papers  
  data-gen/              - v2 data generation and noise reanalysis  
    anomaly_generator.py  - template deformation + interpolation  
    generate_custom_dataset.py - v2 generator (AR(1) noise, 3 templates)  
    analyze_noise_model.py - AR(p), Ljung-Box, drift, stability  
    labeling_view.py      - hand-labelling helper  
    visualize_anomalies_templates.py  
    test_generator_deformation.py  
  anomalies/             - 3 v2 template CSV files  
  data/                  - real recordings + hand labels + v2 synth CSV  
  logs/                  - PNG/HTML inspection plots  
  architectures/         - models, training, inference (v2-trained)  
    train.py              - shared training pipeline (selects model)  
    visualize_inference.py - synthetic-test event-level evaluation  
    inference_real_data.py - real-data deployment trial  
    robustness_test.py    - noise/deformation sweep  
    inference_engine.py    - sliding-window + event matching  
    model_selection.py     - model registry & CLI selector  
    saved_models/         - 1d_cnn.py, resNet.py, cnn_attention.py  
    logs/                  - per-model training/inference outputs  
  pole-anomaly-old/      - v1 generator + v1-trained models  
    data-gen/             - 6 templates, IID Gaussian noise generator  
    architectures/        - v1-trained model checkpoints + logs  
  documentation/         - this report  
    main.tex, preamble.tex, sections/*.tex  
    figures/build_figures.py - reproduces all PNGs in this report
```

Listing A.2. Top-level project structure.

To rebuild every figure in this report from scratch:

```
# Generate v2 synthetic dataset (~10 minutes; 600 MB)  
py data-gen/generate_custom_dataset.py  
  
# (Optional: re-run noise analysis)  
py data-gen/analyze_noise_model.py  
  
# (Optional: re-train v2 models)  
py architectures/train.py --model 1d_cnn  
py architectures/train.py --model resnet  
py architectures/train.py --model cnn_attention  
  
# (Optional: re-run synthetic-test and real-data inference)  
py architectures/visualize_inference.py --model 1d_cnn  
py architectures/inference_real_data.py --model 1d_cnn  
py architectures/robustness_test.py --model 1d_cnn  
# ... same for resnet, cnn_attention  
  
# Rebuild the report's figures (uses both new and old logs)  
py documentation/figures/build_figures.py  
  
# Compile the report  
cd documentation && pdflatex main && pdflatex main
```

A.4 Software environment

All experiments were run on a single laptop with the following software stack:

- Windows 11, Python 3.13.
- PyTorch 2.12 (CUDA 12.8), running on an NVIDIA RTX 5060 Laptop GPU.
- NumPy, Pandas, Matplotlib, SciPy, scikit-learn, statsmodels for data handling, statistics and reporting.
- Plotly for the interactive HTML diagnostics shipped under `logs/`.

A pinned dependency list is included in the top-level `requirements.txt` of the repository.